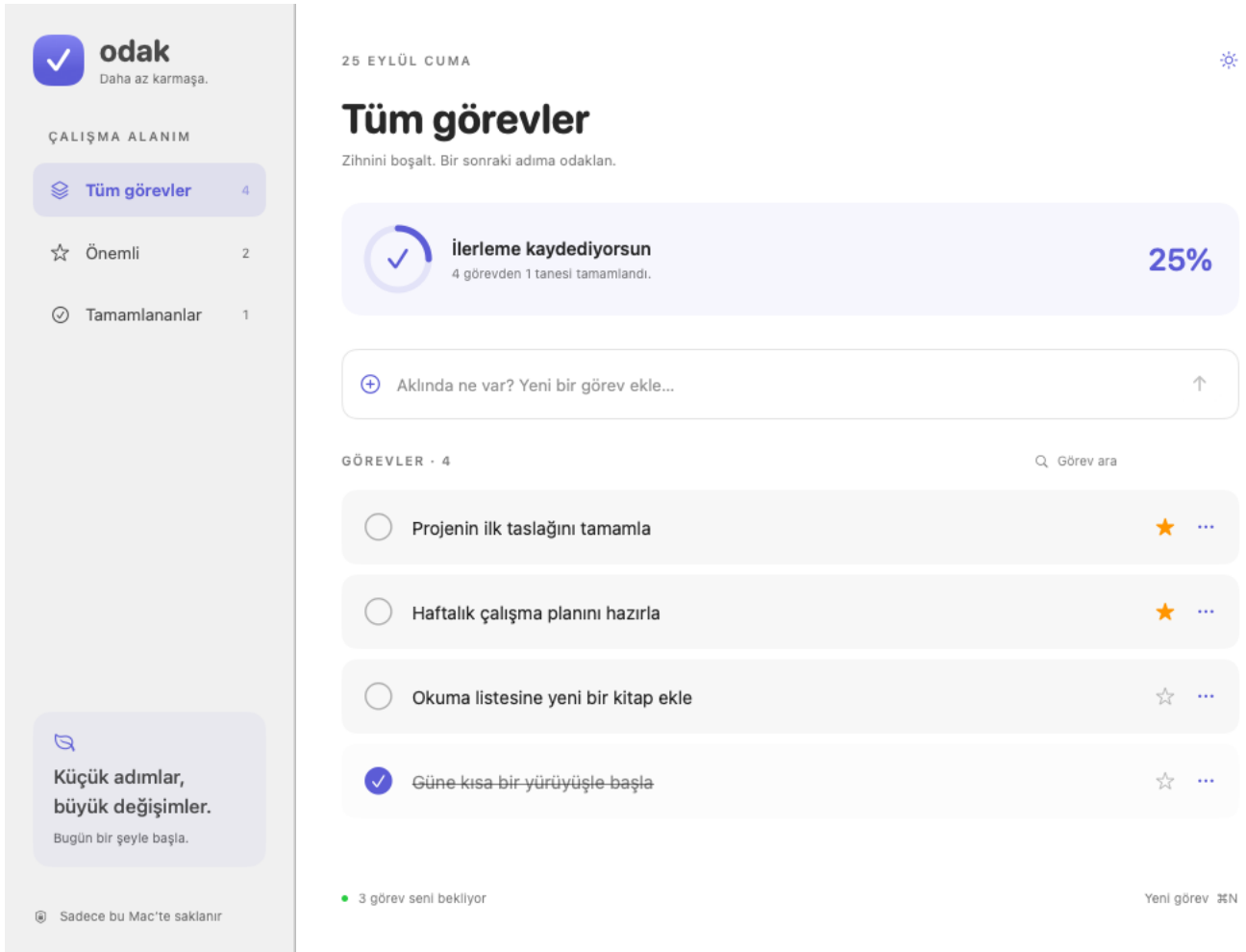


ODAK

Satır Satır Geliştirici Rehberi

SwiftUI ile bir macOS uygulamasını anlamak

Ne yapıyor? Neden böyle yazılmış? Hangi araç hangi sorunu çözüyor?



6 Swift dosyası • 514 kaynak satırı • Yerleşim, veri akışı, kayıt ve testler

Türkçe eğitim belgesi • Kaynak anlık görüntüsü: 25 Eylül 2026

İçindekiler

1. Bu rehberi nasıl okumalı?	4
1.1. Öğrenme hedefleri	4
2. Uygulamanın büyük resmi	5
2.1. Dosyalar ve sorumluluklar	6
2.2. Veri akışı	6
2.3. Somut senaryo: Önemli listesinden görev eklemek	6
3. Kullanılan araçlar ve nedenleri	8
3.1. Çalıştırma ve doğrulama	8
4. SwiftUI yerleşimi: hangi Stack, neden?	10
4.1. HStack: yatay ilişki	10
4.2. VStack ve ZStack	10
4.3. Spacer, frame ve padding birbirinin yerine geçmez	10
4.4. Modifier sırası neden önemlidir?	11
4.5. ScrollView, LazyVStack ve ForEach	11
5. Swift dili ve durum yönetimi	12
5.1. Sık görülen sözdizimi	12
5.2. Durumun sahibi kim?	12
5.3. Closure ve fonksiyon değeri	13
5.4. MainActor ne sağlar, ne sağlamaz?	13
6. Kalıcı veri, hatalar ve tasarım kararları	14
6.1. Kayıt işleminin sırası	14
6.2. JSON ve model uyumu	14
6.3. Mevcut davranışın sınırları	14

7. Kaynak dosyaları satır satır	16
7.1. ios_testApp.swift	17
7.2. TaskStore.swift	19
7.3. ContentView.swift	24
7.4. ios_testTests.swift	43
7.5. ios_testUITests.swift	46
7.6. ios_testUITestsLaunchTests.swift	49
8. Xcode ayarları ve yardımcı dosyalar	51
8.1. Applcon Contents.json nasıl okunur?	51
8.2. .gitignore dosyasının beş satırı	52
9. Testleri nasıl yorumlamalı?	53
10. Öğrenmeyi pekiştiren uygulamalar	54
10.1. Yerleşim deneyi	54
10.2. Yeni bir filtre	54
10.3. Önemli işaretinin kalıcılığı	54
10.4. Hata sonrası yeniden deneme tasarımı	54
10.5. UI testini büyütme	54
10.6. Kısa öz değerlendirme	54
11. Resmî kaynaklar ve belge üretimi	56
11.1. PDF'nin nasıl üretildiği	56
11.2. Kaynak anlık görüntüsü ve kapsam kontrolü	56

1. Bu rehberi nasıl okumalı?

Bu belge, temel programlama bilgisi olan fakat SwiftUI'ya yeni başlayan bir yazılımcı için hazırlanmıştır. Amaç yalnızca kodu Türkçeye çevirmek değil; veri, kullanıcı eylemi, yerleşim ve kalıcı kayıt arasındaki ilişkiyi açıklamaktır. Projenin adı dosyalarda `ios_test` olsa da çalıştırılan hedef macOS uygulamasıdır.

Kapsam: `ios_testApp.swift`, `TaskStore.swift`, `ContentView.swift` ve üç test dosyasının toplam 514 fiziksel satırı. Her satır özgün numarasıyla gösterilir; boş satırlar, yorumlar ve kapanışlar da atlanmaz. Xcode'un ürettiği `project.pbxproj` içindeki yüzlerce nesne kaydı ise satır satır değil, geliştirici açısından önemli ayarlar üzerinden açıklanır. Görseller ve JSON asset kayıtları ayrıca ele alınır.

Satır açıklamalarındaki “neden” ifadeleri, mevcut kodun sağladığı faydaya dayalı teknik yorumlardır. Özgün yazarın belgelenmemiş düşüncesini bildiğimiz anlamına gelmez. Açılıştaki Dock simgesi yenilemesinin amacı ise kaynak kod yorumunda açıkça belirtilmiştir.

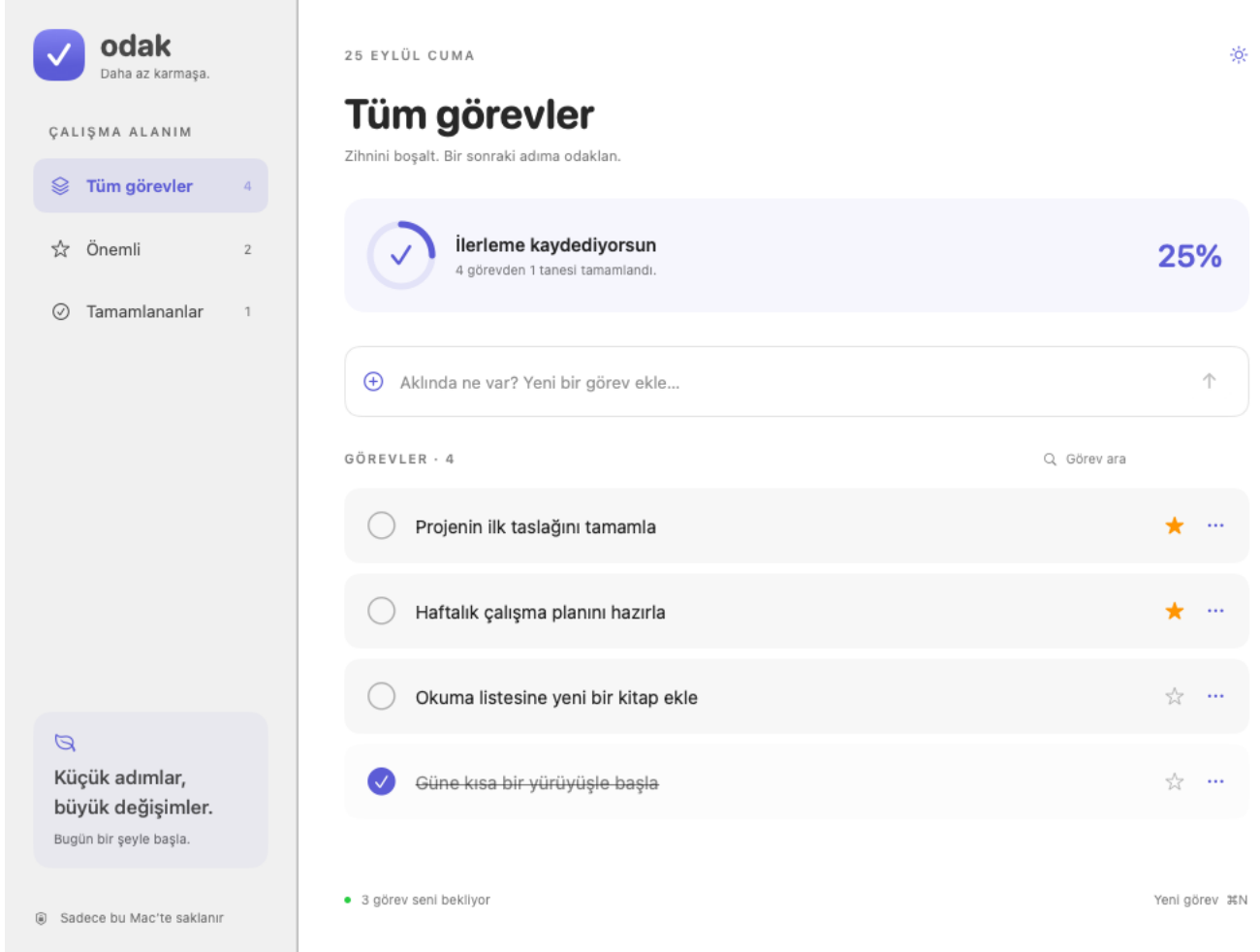
Önerilen sıra: önce 2–6. bölümlerde kavramları öğren; sonra uygulama girişini, veri deposunu ve arayüzü bu sırayla oku. Testleri en son okuyarak uygulamanın hangi davranışlarının gerçekten doğrulandığını gör. Kaynak kod değişikçe satır numaraları da değişir; bu PDF, üretildiği andaki dosyaların bir anlık görüntüsüdür.

1.1. Öğrenme hedefleri

- Bildirimsel arayüzün, elle görünüm güncellemekten farkını açıklamak.
- HStack, VStack ve ZStack arasında görsel ihtiyaca göre seçim yapmak.
- Geçici arayüz durumu ile kalıcı görev verisini birbirinden ayırmak.
- Bir düğmeye basılmasından JSON kaydına ve ekranın yenilenmesine kadar akışı izlemek.
- Closure, binding, optional, guard, protocol ve actor kavramlarını gerçek satırlarda tanımak.
- Bir testin neyi kanıtladığını ve neyi kanıtlamadığını ayırt etmek.

2. Uygulamanın büyük resmi

Odak, görev ekleme, önemli işaretleme, tamamlama, arama, filtreleme, düzenleme ve silme sunan küçük bir masaüstü uygulamasıdır. Uygulamada sunucu, oturum açma, ağ isteği veya bulut eşitlemesi bulunmaz. Kalıcı veri bir JSON dosyasındadır; arama metni ve seçili filtre gibi ekran tercihleri bu dosyaya yazılmaz.



Odak açık tema: arayüzün genel görünümü

2.1. Dosyalar ve sorumluluklar

Parça	Sorumluluğu	Burada yapılmayan iş
ios_testApp.swift	Uygulamayı başlatır, pencereyi kurar, ortak TaskStore'u üretir.	Görev satırlarını çizmez.
TodoItem	Tek görevin kimliğini, başlığını, durumlarını ve tarihini tutar.	Dosyaya kendi kendine yazmaz.
TaskStore	Görev işlemlerini doğrular, JSON okur/yazar, değişikliği yayımlar.	Pencere veya düğme yerleşimi bilmez.
ContentView	Arayüzü, filtreyi, aramayı, giriş odağını ve düzenleme formunu yönetir.	JSON kodlama ayrıntısını uygulamaz.
ios_testTests	Depolama ve model davranışlarını geçici dosyalarla sınar.	Gerçek fare/klavye etkileşimi yapmaz.
ios_testUITests	Uygulama açılışı, performans ölçümü ve görüntü eki üretir.	Bütün özellikleri uçtan uca doğrulamaz.

Bu yapı, görünüm ile veri işlemlerini ayırır. TaskStore hem gözlemlenebilir durum hem depolama sorumluluğu taşır; ContentView içinde de filtreleme mantığı vardır. Dolayısıyla projeyi katı, çok katmanlı bir MVVM uygulaması diye sunmak yerine, küçük uygulamaya uygun bir görünüm + ortak veri deposu tasarımı olarak okumak daha isabetlidir.

2.2. Veri akışı

```

ios_testApp
  oluşturur: @StateObject TaskStore
  paylaşır: .environmentObject(store)
    |
    v
ContentView: @EnvironmentObject store
  kullanıcı eylemi -> store.add / rename / toggle / delete
    |
    v
TaskStore.update
  doğrula -> kopyayı değiştir -> JSON yaz -> items değerini yayımla
    |
    v
SwiftUI, değişen duruma bağımlı arayüzü yeniden değerlendirir

```

TaskStore'un tek örneği uygulama düzeyinde sahiplenilir. WindowGroup üzerinden birden fazla pencere oluşturulursa görev deposu ortaktır. Her ContentView'in arama ve filtre gibi @State değerleri ise kendi görünüm kimliğine bağlı yerel durumudur. Ortak veri ile pencereye özgü sunum böyle ayrılır.

2.3. Somut senaryo: Önemli listesinden görev eklemek

1. Kullanıcı Önemli listesini seçer; filter state'i .important olur.
2. TextField'a yazılan metin, \$newTitle üzerinden newTitle değerine aktarılır.
3. Return veya ok düğmesi addTask() çağırır.

4. `addTask`, `important: true` ile `store.add` çağırır.
5. `TaskStore` başlığın dış boşluklarını temizler, boş başlığı reddeder.
6. `update`, dizinin kopyasına yeni görevi başa ekler ve JSON dosyasına yazar.
7. Yazma başarılıysa items güncellenir; hata varsa eski liste korunur.
8. Başarı yolunda taslak ve arama temizlenir, giriş alanı yeniden odaklanır.
9. Liste, sayaçlar ve ilerleme kartı kendi hesaplarını güncel durumdan üretir.

3. Kullanılan araçlar ve nedenleri

Araç / teknoloji	Projedeki rolü	Neden uygun?
Swift	Uygulama ve test dili	Tip güvenliği, optional ve protokollerle niyet açık yazılır.
SwiftUI	Bildirimsel arayüz çatısı	Durumdan arayüz üretir; küçük bileşenler birleştirilir.
AppKit	macOS platform API'leri	Dock simgesi ve yerel renk gibi masaüstüne özgü işleri sağlar.
Foundation	URL, Date, UUID, Data, FileManager, JSON	Küçük uygulama için gerekli sistem veri ve dosya araçları hazırdır.
ObservableObject / Published	Gözlemlenebilir durum	SwiftUI'ya görev verisinin değiştiğini bildirir; projede ayrı bir Combine iş hattı yazılmaz.
SF Symbols	Sistem simgeleri	Kontroller için ayrı PNG dosyaları taşımadan tutarlı simgeler sağlar.
Xcode / Canvas	Derleme, çalıştırma, önizleme	Apple platform hedeflerini ve SwiftUI önizlemesini yönetir.
Swift Testing	Model testleri	@Test ve beklenti makrolarıyla kısa davranış testleri yazılır.
XCTest / XCUITest	UI ve performans testleri	Çalışan uygulamayı başlatma, ölçme ve ekran görüntüsü alma imkânı sağlar.
Git	Kaynak sürüm takibi	Kod ve dokümantasyon değişiklikleri karşılaştırılabilir.

Uygulama kaynaklarında üçüncü taraf paket bağımlılığı yoktur. JSON saklama, bu ölçekte ayrı bir veritabanı kurma gereksinimini azaltır. Karşılığında her değişiklikte bütün görev dizisi yeniden kodlanır ve yazılır. Büyük veri, sorgu, ilişkiler veya eşitleme gereksinimi doğarsa depolama tasarımı yeniden değerlendirilmelidir.

Proje minimum macOS 14 hedefler. SWIFT_VERSION = 5.0 ayarı Swift 5 dil uyumluluk modunu ifade eder; kullanılan derleyicinin kesin olarak Swift 5 olduğu anlamına gelmez. Kaynakta #Preview, switch ifadeleri ve Swift Testing gibi daha yeni araç zinciri özellikleri bulunduğu için tam Xcode 16 veya üzeri geliştirme ortamı uygundur.

3.1. Çalıştırma ve doğrulama

Xcode ile ios_test.xcodeproj açılır, ios_test şeması ve My Mac hedefi seçilir. Command+R uygulamayı, Command+U testleri çalıştırır. Canvas, ContentView'in #Preview bloğunu kullanır. Command Line Tools seçiliyse xcodebuild komutları tam Xcode'a yönlendirilmelidir; bu belge üretimi sistemdeki geliştirici dizini seçimini değiştirmez.

```
xcodebuild test \
  -project ios_test.xcodeproj \
  -scheme ios_test \
  -destination 'platform=macOS'
```

Bu rehber için uygulama testleri yeniden çalıştırılmadı. Test bölümleri mevcut test kodunun statik incelemesidir; bir testin bulunması veya burada açıklanması, son koşulda geçtiğinin kanıtı değildir.

4. SwiftUI yerleşimi: hangi Stack, neden?

SwiftUI’da bir görünümün body özelliği, o anki duruma göre nasıl görünmesi gerektiğini tarif eder. Geliştirici her metin değişikliğinde etiketi elle güncellemez. SwiftUI, kimlik ve bağımlılık bilgisiyle görünüm tanımlarını yeniden değerlendirir ve gerekli ekran güncellemelerini uygular. body’yi tek sefer çalışan bir çizim fonksiyonu gibi düşünmek yanıltıcıdır.

4.1. HStack: yatay ilişki

HStack çocuklarını yatay ekseninde yerleştirir. Odak’ın dış HStack’i sidebar ile ana paneli yan yana koyar. Görev satırındaki HStack ise daire, başlık, yıldız ve menüyü tek satırda toplar. Birimler birbirinin yanında anlamlı bir bütün oluşturduğu için yatay yerleşim seçilmiştir. Apple’ın HStack referansı için kaynak [1]’e bak.

```
HStack(spacing: 13) {
    // Tamamlama kontrolü
    // Esnek görev başlığı
    // Önemli düğmesi
    // İşlem menüsü
}
```

HStack’i VStack ile değiştirirsen aynı kontroller alt alta dizilir; ZStack ile değiştirirsen aynı bölgede üst üste yerleşir. Yani Stack seçimi bir hızlandırma numarası değil, öğeler arasındaki geometrik ilişkinin tarifidir. HStack tek başına otomatik satır kaydırma yapmaz; dar alanda metin ve kontrollerin sığması ayrıca düşünülmelidir.

4.2. VStack ve ZStack

VStack dikey gruplar için kullanılır: ana panelde header, progressCard, composer, taskList ve footer yukarıdan aşağıya gelir. alignment: .leading yatay hizayı, spacing: 26 ise komşu çocuklar arasındaki dikey uzaklığı belirler. “Dikey yığın” içindeki alignment’ın yine yatay bir hizayı belirlemesi sık karıştırılan noktadır.

ZStack katmanları üst üste bindirir. İlerleme halkasında soluk tam çember, mor kısmı çember ve ortadaki onay simgesi aynı merkezi paylaşır. Üçü aynı görselin katmanları olduğundan ZStack kullanılır. Arka plan yapmak için her zaman ZStack gerekmez; tek görünümün arkasına şekil eklemek için background daha doğrudan olabilir.

```
HStack: [ simge ] [ başlık ] [ yıldız ]

VStack: [ başlık ]
        [ açıklama ]
        [ giriş alanı ]

ZStack: [ arka halka + ilerleme yayı + merkez simgesi ]
```

4.3. Spacer, frame ve padding birbirinin yerine geçmez

Spacer, içinde bulunduğu yığının ana eksenindeki uygun boşluğu alır. HStack içinde sağdaki öğeleri sona iter; VStack içinde alt içeriği aşağı iter. Bu projede sidebar içindeki Spacer, motivasyon kartını aşağı taşıırken footer içindeki Spacer kısayolu sağa taşır.

frame, görünümün yerleşimde talep ettiği alanı ve o alan içindeki hizasını etkiler. maxWidth: .infinity, ebeveynin verdiği uygun genişliğe yayılma isteğidir; sınırsız fiziksel ekran alanı oluşturmaz. width: 226 ise sidebar için belirli bir genişliktir. Gerçek pencere boyutu, ebeveynin boyut önerileri ve minimum sınırlar birlikte sonuç üretir.

padding, bir görünümün çevresine boşluk ekler. spacing, bir yığının çocukları arasındaki mesafedir. Aynı tasarımda ikisi birlikte kullanılabilir. Örneğin progressCard'daki HStack(spacing: 18) kart öğelerini ayırır; .padding(20) bütün kartın içeriğini dış sınırdan uzaklaştırır.

4.4. Modifier sırası neden önemlidir?

Modifier'lar zincir halinde yeni bir görünüm değeri oluşturur. Sıra özellikle boyut ve arka plan ilişkisini değiştirir. Kaynakta önce padding sonra background kullanılması, arka planın boşluk eklenmiş alanı da kaplamasını sağlar. Aşağıdaki örnekler eğitim amaçlıdır; uygulamaya eklenmiş kod değildir.

```
Text("Odak")
    .padding(16)
    .background(Color.purple)
// Mor alan metin + boşluğu kapsar.

Text("Odak")
    .background(Color.purple)
    .padding(16)
// Mor alan metnin arkasındadır; dış boşluk mor olmaz.
```

foregroundColor içerik çizimini, background içerik arkasını, clipShape ise çizimin görünür sınırını etkiler. tint, onu kullanan alt kontrollere vurgu rengi sağlar. Bu dört API aynı şeyi yapmaz. Mevcut projede mor tint düğmelere yön verirken, önemli yıldız açıkça turuncu foregroundStyle kullanır.

4.5. ScrollView, LazyVStack ve ForEach

ScrollView kaydırma davranışını, LazyVStack dikey ve ihtiyaç oldukça üretilen içeriği, ForEach ise veriden kimlikli görünüm üretimini sağlar. Birlikte çalışırlar ama sorumlulukları ayrıdır. TodoItem.id, görev başlığı değişse veya sırası farklılaşsa da satırın aynı görev olarak tanınmasını sağlar. Kaynak [2], tembel dikey yerleşimin API referansıdır.

Bu projede tembel olan görünüm üretimidir. visibleItems hesaplaması yine görev dizisini filtreler ve sıralar; JSON dosyasının tamamı belleğe alınır. Dolayısıyla LazyVStack eklemek, veri işleme maliyetlerinin tamamını ortadan kaldırmaz. Küçük liste için basitlik uygundur; büyük listede ölçümle karar verilmelidir.

5. Swift dili ve durum yönetimi

5.1. Sık görülen sözdizimi

Yazım	Anlamı	Projedeki örnek
let / var	Yeniden atanamayan / atanabilen bağ	accent sabit, filter değişken.
struct / class	Değer türü / referans türü	TodoItem verisi, TaskStore ortak nesnesi.
enum	Sınırlı olası durumlar	TaskFilter üç filtreyi sınırlar.
private / private(set)	Erişim sınırı / sadece setter sınırı	items dışarıdan okunur, doğrudan yazılamaz.
String?	Değer veya nil	storageError, hata yokken nil.
if let / guard let	Optional'ı güvenli açma	Hata metni ve görev indeksi.
??	nil için varsayılan değer	Özel URL yoksa Application Support.
\$newTitle	Property wrapper'ın yansıtılan değeri	TextField'a Binding<String> verir.
\$0, \$1	Closure'ın kısa parametre adları	filter ve sorted içindeki öğeler.
\.isCompleted	Bir özelliğe key path	Tamamlananları filtrelemek.
&updated / inout	Değiştirilebilir argüman aktarımı	Ortak update closure'ı.
some View	Gizli somut dönüş tipi	Görünüm bileşenlerinin dönüşü.
@Test / #expect	Özellik işareti / test makrosu	Swift Testing testleri.

\$ sözdiziminin iki farklı rolüne özellikle dikkat et: \$newTitle bir binding elde eder; \$0 closure'ın ilk argümanına verilen kısa addır. Benzer görünmeleri aynı mekanizma oldukları anlamına gelmez. String içindeki \ (değer) ise string interpolation'dır; değeri metne dönüştürüp birleştirir.

5.2. Durumun sahibi kim?

Mekanizma	Bu projede nerede?	Görevi
@StateObject	ios_testApp.store	Gözlemlenebilir deponun yaşamını sahiplenir.
.environmentObject	App ve Preview	Aynı nesneyi alt görünüm ağacına sağlar.
@EnvironmentObject	ContentView.store	Sağlanan nesneyi alır ve değişikliklerini izler.
@Published	items, storageError	Nesnenin veri değişikliğini gözlemcilere bildirir.
@State	filter, search, taslaklar	Görünümüne ait geçici değer durumunu saklar.
@FocusState	composerFocused	Durumu klavye odağıyla bağlar.
Binding	\$search, \$editedTitle	Bir değere iki yönlü okuma-yazma bağlantısı sağlar.

@StateObject seçimi, her body değerlendirmesinde yeni TaskStore üretme hatasını önler. @EnvironmentObject ise nesnenin sahibi değildir; nesne daha yukarıda sağlanmalıdır. Kaynak [3] sahiplik modelini, kaynak [4] model verisinin görünüm ağacında paylaşımını açıklar. Proje mevcut ObservableObject yaklaşımını kullanır; bu rehber kodu yeni bir gözlem modeline dönüştürmez.

Hesaplanan completedCount ve visibleItems ayrı @State değildir. Bunlar asıl veriden türetilir. Aynı bilginin hem items hem filteredItems gibi iki bağımsız yerde tutulması, birini güncelleyip diğerini unutma riskini artırır. Mevcut tasarım küçük listelerde anlaşılırdır; görünür liste her kullanımda yeniden hesaplanabildiği için büyüyen veriyle maliyeti ölçmek gerekir.

5.3. Closure ve fonksiyon değeri

Closure, argüman olarak taşınabilen kod bloğudur. TaskStore.update bir işlem alır: eklemede insert, silmede removeAll, düzenlemede başlık ataması yapılır. Kayıt kodu ortak kalır. Swift closure kuralları için kaynak [5]'e bak.

```
update { items in
    // items üzerinde istenen değişiklik
}
```

inout [TodoItem], closure'ın bu dizi değişkenini değiştirebilmesini sağlar. update içindeki mutation(&updated) çağrısında & gereklidir. Bu bir ağ isteği, async iş veya sonradan çağrılan kuyruk değildir; mevcut closure senkron çalışır. Dış update ile iç closure'ın return kapsamlarının farklı olması önemlidir: indeksi bulamayan closure'dan return etmek, dış fonksiyonun devamını otomatik durdurmaz.

5.4. MainActor ne sağlar, ne sağlamaz?

TaskStore'un @MainActor olması, durumuna erişimi ana aktör bağlamında toplar. SwiftUI'yla ilişkili değişikliklerin tutarlı bağlamda yapılmasına yardımcı olur. Buna karşın dosya I/O'su otomatik olarak arka plana taşınmaz. Data(contentsOf:) ve write çağrıları burada senkrondur. Küçük JSON için basitlik sağlar; büyük dosyada kullanıcı arayüzü takılabilir. Kaynak [6], MainActor ve actor izolasyonunun temelini açıklar.

6. Kalıcı veri, hatalar ve tasarım kararları

6.1. Kayıt işleminin sırası

```

items (son başarılı durum)
|
v
updated = items -> mutation(&updated)
|
v
JSONEncoder -> atomik dosya yazımı
|
+-- başarı --> items = updated --> arayüz güncellemesi
|
+-- hata ----> storageError      --> eski items korunur

```

Bu sıra, ekranda başarılı görünmüş ama diske yazılmamış bir değişikliği normal başarı yolu olarak göstermemek için seçilmiştir. Önce items değişseydi ve yazma başarısız olsaydı, geri alma veya tutarsızlığı yönetme gereksinimi doğardı. Buradaki bedel, dosya yazmasının UI güncellemesinden önce senkron tamamlanmasıdır.

.atomic seçeneği bir yardımcı dosyaya yazıp hedefi değiştirme yaklaşımıyla yarım dosya riskini azaltır. “Atomik” demek otomatik yedek, şifreleme, her koşulda veri kurtarma veya çok süreçli veritabanı işlemi demek değildir. İlgili Foundation seçeneği kaynak [7]’dedir.

6.2. JSON ve model uyumu

Codable, TodoItem’in saklanan alanlarından JSON kodlama ve çözümleme üretir. Date ve UUID de bu mekanizmaya katılır. Varsayılan JSONEncoder tarih stratejisi, insanın okuyacağı ISO 8601 metni olarak ayrıca ayarlanmamıştır; tarih çıktısını özel format sanmamak gerekir. id ve createdAt için başlangıç değeri bulunması, eski JSON’da eksik zorunlu alanların otomatik olarak doldurulacağı anlamına gelmez.

İleride alanlar eklenirse dosya formatı uyumluluğu düşünülmelidir: optional alan, özel init(from:), varsayılan çözümleme veya şema geçişi seçenekleri değerlendirilebilir. Mevcut uygulamada şema sürümü ve veri göçü mekanizması yoktur. createdAt şu anda saklanır ama ekranda ve sıralamada kullanılmaz.

6.3. Mevcut davranışın sınırları

- Okuma hatası, yeni değişiklikleri durdurur ve mevcut dosyanın üzerine yazılmasını önler. Bozuk JSON’u otomatik düzeltmez.
- Yazma hatası da storageError alanını doldurur; update başındaki guard sonraki değişiklikleri engeller. Arayüzde bir “yeniden dene ve hatayı temizle” akışı yoktur.
- Silme doğrudan yapılır. destructive rolü onay penceresi veya undo sağlamaz.
- Hata mesajları, gerçek Error ayrıntılarını kullanıcıya veya günlüğe taşıyor. Sorun tanılama sınırlıdır.
- Görevlerin yerel saklanması, dosyanın şifrelenmiş olduğu anlamına gelmez. Uygulama seviyesinde şifreleme uygulanmaz.
- Date.now kullanan başlık bir saat/takvim zamanlayıcısı değildir. Güneş simgesi hava durumu göstergesi değildir.

- UI testleri gerçek kullanıcı görev dosyasından ayrılmış bir launch argümanı kullanmıyor. Etkileşim testleri genişletilmeden önce yalıtılmış test deposu sağlanmalıdır.

Bunlar bu rehberde düzeltilen hatalar veya eklenen özellikler değildir; mevcut kodu doğru anlamak için kaydedilmiş gözlemlerdir. Ürün gereksinimi ve ölçüm olmadan her küçük uygulamaya veritabanı, soyutlama katmanları veya ağ sistemi eklemek gerekmez.

7. Kaynak dosyaları satır satır

Sonraki altı bölümde her fiziksel kaynak satırı gösterilir. Numaralar ilgili dosyanın kendi satır numaralarıdır. Uzun kod satırları PDF’de birkaç görsel satıra sarılabilir; bu durum kaynak numarasını değiştirmez. Boş satırların düzenleme amacı, kapanışların ise hangi kapsamı bitirdiği belirtilir. Aynı satırda birden fazla modifier veya ifade varsa birlikte açıklanır.

Satır açıklamalarında “kapsam” sözcüğü, süslü parantezle açılan fonksiyon, closure, tip veya kontrol bloğunu ifade eder. Kapanış açıklamasındaki başlangıç numarası, ilgili dosyanın açılış satırına geri dönmeyi kolaylaştırır.

7.1. ios_testApp.swift

ios_test/ios_testApp.swift

Giriş noktası, macOS delege köprüsü, ortak veri deposu ve pencere yapılandırması.

```
001 import SwiftUI
```

SwiftUI modülünü içe aktarır. App, Scene, WindowGroup ve özellik sarmalayıcıları bu çatıdan gelir; uygulamanın bildirimsel arayüz girişini yazabilmek için gerekir.

```
002 import AppKit
```

AppKit, macOS'un yerel uygulama ve pencere API'lerini sağlar. Burada NSApplication, NSImage ve uygulama delegesi için kullanılır; bu bağımlılık kodun doğrudan iOS'ta çalışmamasının nedenlerinden biridir.

003 Boş satır — kod bölümlerini ayırır.

```
004 final class OdakAppDelegate: NSObject, NSApplicationDelegate {
```

NSObject'tan türeyen ve NSApplicationDelegate protokolüne uyan bir referans türü tanımlar. final, alt sınıf oluşturulmasını kapatır. SwiftUI yaşam döngüsüne macOS'a özgü bir açılış davranışı eklemek için küçük bir köprü görevi görür.

```
005 func applicationDidFinishLaunching(_ notification: Notification) {
```

macOS, uygulama açılışı tamamlandığında bu delege metodunu çağırır. Notification parametresi protokol imzasının parçasıdır; gövdede kullanılmaz.

```
006 // Refresh the running app's Dock icon even when Launch Services has
```

Yorum, önceki geliştirme derlemesinden kalmış Dock simgesini yenileme gerekçesini açıklar.

```
007 // retained the placeholder from an earlier development build.
```

Yorum, önceki geliştirme derlemesinden kalmış Dock simgesini yenileme gerekçesini açıklar.

```
008 guard let iconURL = Bundle.main.url(forResource: "AppIcon", withExtension: "icns"),
```

Bundle.main içinden Applcon.icns dosyasının URL'sini arar. guard let, bulunamayan kaynağın optional sonucunu güvenli biçimde ele alır; dosya adı asset kataloğundaki Applcon ile ilişkilidir.

```
009 let icon = NSImage(contentsOf: iconURL) else { return }
```

URL'deki dosyayı NSImage olarak yükler. URL ya da görüntü üretilemezse return ile sessizce çıkar; simge yenileme hatası uygulamanın açılışını engellemez.

```
010 NSApplication.shared.applicationIconImage = icon
```

Çalışan uygulamanın Dock simgesini yüklenen görselle günceller. Kaynak yorumuna göre amaç, Launch Services'in önceki geliştirme derlemesinden tuttuğu geçici simgeyi yenilemektir; kalıcı bir önbellek temizleme işlemi değildir.

```
011 } — 5. satırda açılan kapsamı kapatır: func applicationDidFinishLaunching(_ notification: Notification) {
```

```
012 } — 4. satırda açılan kapsamı kapatır: final class OdakAppDelegate: NSObject, NSApplicationDelegate {
```

013 Boş satır — kod bölümlerini ayırır.

```
014 @main
```

@main bu tipi programın giriş noktası yapar. SwiftUI, App protokolü üzerinden uygulama yaşam döngüsünü başlatır; ayrıca elle bir main() yazılmaz.

```
015 struct ios_testApp: App {
```

ios_testApp, SwiftUI App protokolüne uyar. Tipin adı teknik hedef adıdır; kullanıcıya görünen pencere adı aşağıda Odak olarak belirlenir.

```
016 @NSApplicationDelegateAdaptor(OdakAppDelegate.self) private var appDelegate
```

NSApplicationDelegateAdaptor, SwiftUI yaşam döngüsüne OdakAppDelegate örneğini bağlar. Böylece delegenin açılış metodu gerçekten çağrılır; sınıfı yalnızca tanımlamak yeterli olmazdı.

```
017 @StateObject private var store = TaskStore()
```

TaskStore'un sahibi uygulamadır. @StateObject aynı uygulama kimliği boyunca nesnenin yaşamını SwiftUI'ya yönetir; body yeniden hesaplanırken her seferinde yeni depo oluşturulmaz. Pencere bu ortak nesneyi kullanır.

018 Boş satır — kod bölümlerini ayırır.

```
019 var body: some Scene {
```

body, gösterilecek sahneleri tarif eder. some Scene, somut dönüş tipini gizler ama derleyicinin tip bilgisini korur; burada View yerine Scene kullanılır çünkü uygulama düzeyinde pencere tanımlanır.

```
020 WindowGroup("Odak") {
```

Odak başlıklı bir WindowGroup oluşturur. macOS pencere yaşam döngüsünü SwiftUI yönetir; grup birden fazla pencere açılmasına da uygun bir sahne türüdür.

```
021 ContentView()
```

Ana arayüzün kök görünümünü oluşturur. ContentView, kendi kalıcı deposunu burada üretmez; bir sonraki satırda dışarıdan sağlanır.

```
022 .environmentObject(store)
```

store nesnesini görünüm ağacının environment alanına koyar. ContentView içindeki @EnvironmentObject bu aynı örneği bulur; bu bağlantı kaldırılırsa görünümün depo bağımlılığı karşılanmaz.

```
023 .environment(\.locale, Locale(identifier: "tr_TR"))
```

Locale environment değerini tr_TR yapar. Yerelleştirmeye duyarlı alt görünüm Türkçe yerel ayarını kullanabilir; bu satır tek başına bütün metinleri otomatik çevirmez, metinler zaten Türkçe yazılmıştır.

024 } — 20. satırda açılan kapsamı kapatır: WindowGroup("Odak") {

```
025 .defaultSize(width: 1000, height: 740)
```

Yeni pencerenin başlangıç boyutunu 1000 × 740 punto önerir. Bu bir sabit boyut kilidi değildir; kullanıcı pencereyi büyütüp küçültebilir.

```
026 .windowStyle(.hiddenTitleBar)
```

Başlık çubuğunu gizleyen pencere stilini seçer. Daha sade bir içerik alanı sağlar; uygulamanın bütün pencere davranışlarını ortadan kaldırmaz.

```
027 .windowResizability(.contentMinSize)
```

Pencerenin küçültülebileceği alt sınırı içeriğin minimum boyutuna bağlar. ContentView'deki 800 × 580 minimum frame ile birlikte çalışır.

028 } — 19. satırda açılan kapsamı kapatır: var body: some Scene {

029 } — 15. satırda açılan kapsamı kapatır: struct ios_testApp: App {

7.2. TaskStore.swift

ios_test/TaskStore.swift

Değer modeli, güvenli değişiklik akışı ve atomik JSON kaydı. Önce bu dosyayı anlamak, arayüzün veri çağrılarını okumayı kolaylaştırır.

```
001 import SwiftUI
```

SwiftUI içe aktarılır. ObservableObject ve @Published ile SwiftUI'nin kullandığı gözlemlenebilir veri modeli kurulur. Foundation tipleri de bu modülün aktardığı API'ler üzerinden erişilebilir durumdadır.

002 Boş satır — kod bölümlerini ayırır.

```
003 struct TodoItem: Identifiable, Codable, Equatable {
```

TodoItem bir değer türüdür. Identifiable, liste satırlarını id ile ayırt eder; Codable, JSON kodlama ve çözümlemeyi sentezler; Equatable, alan değerleri üzerinden eşitlik karşılaştırmasını mümkün kılar. Bu protokoller farklı sorumluluklar taşır.

```
004 var id = UUID()
```

Her yeni göreve UUID kimliği verir. Başlığı aynı olan iki görev ayrı kalır; JSON'a yazılan kimlik yeniden yüklenince korunur. Satır kimliği için dizi indeksini kullanmamak sıralama ve silme sırasında yararlıdır.

```
005 var title: String
```

Görev başlığını zorunlu String olarak saklar. Varsayılanı olmadığı için yeni TodoItem oluşturulurken title verilmelidir; boş olup olmadığı model alanında değil TaskStore işlemlerinde doğrulanır.

```
006 var isCompleted = false
```

Yeni görev tamamlanmamış başlar. Bool, iki durumlu bilgiyi tutar; tamamlandı işareti değiştiğinde başlığın çizilmesi ve filtreler bu alanı okur.

```
007 var isImportant = false
```

Yeni görev varsayılan olarak önemli değildir. Oluşturucuya isImportant verilerek bu varsayılan değiştirilebilir; Önemli filtresinden ekleme bunu kullanır.

```
008 var createdAt = Date()
```

Oluşturulma zamanını kaydeder. Mevcut arayüz bu alanı göstermez veya sıralamada kullanmaz; alan Codable nedeniyle yine de JSON içinde yer alır.

009 } — 3. satırda açılan kapsamı kapatır: struct TodoItem: Identifiable, Codable, Equatable {

010 Boş satır — kod bölümlerini ayırır.

```
011 @MainActor
```

TaskStore'u MainActor'a izole eder. Arayüzün gözlemlediği duruma erişimi aynı aktörde toplar; bu, dosya işlemlerini arka plana taşımaz. Mevcut senkron okuma ve yazma ana aktörü meşgul edebilir.

```
012 final class TaskStore: ObservableObject {
```

final class, paylaşılan kimliği olan ve miras alınmayan bir nesne tanımlar. ObservableObject ile değişiklikler SwiftUI tarafından izlenebilir. struct TodoItem veriyi, class TaskStore ise değişen ortak durumu yönetir.

```
013 @Published private(set) var items: [TodoItem] = []
```

Görev dizisi başlangıçta boştur. @Published değişiklik bildirimi üretir; private(set) dış kodun diziyi okuyup doğrudan değiştirememesini sağlar. Böylece değişikliklerin dosyaya kayıt yolundan geçmesi teşvik edilir.

```
014 @Published var storageError: String?
```

İsteğe bağlı hata metnidir; başlangıçta nil olur. @Published sayesinde hata oluştuğunda arayüz mesajı güncellenir. Setter private değildir; mevcut UI bu alanı temizlemez.

```
015 private let fileURL: URL
```

Depolama konumunu değişmez ve sınıfa özel URL olarak tutar. Dosya yolunun tek yerde tutulması tüm işlemlerin aynı dosyaya gitmesini sağlar.

016 Boş satır — kod bölümlerini ayırır.

```
017 init(fileURL: URL? = nil) {
```

Kurucu isteğe bağlı bir dosya URL'si kabul eder. Üretimde varsayılan yol, testlerde geçici yol kullanılır. Bu, bağımlılık enjeksiyonunun basit bir örneğidir ve gerçek kullanıcı verisini testlerden ayırır.

```
018 self.fileURL = fileURL ?? URL.applicationSupportDirectory
```

?? operatörü, verilen URL nil ise Application Support dizinini seçer. self.fileURL nesnenin alanını, sağdaki fileURL ise kurucu parametresini ifade eder.

```
019 .appendingPathComponent("Odak", isDirectory: true)
```

Uygulamaya özgü Odak alt dizinini ekler. isDirectory, URL'nin dizin olarak ele alınacağını belirtir; burada henüz disk üzerinde dizin oluşturulmaz.

```
020 .appendingPathComponent("tasks.json")
```

Görev dosyası adını yola ekler. Sonuç varsayılan olarak kullanıcının Application Support/Odak/tasks.json konumudur.

```
021 guard FileManager.default.fileExists(atPath: self.fileURL.path) else { return }
```

Dosya yoksa kurucudan çıkar ve boş listeyi devam eder. İlk açılışta veri olmaması normal kabul edilir; okuma hatası olarak gösterilmez.

```
022 do {
```

Hata fırlatabilen okuma ve çözümleme işlemlerini do bloğunda toplar. Bir hata oluşursa aşağıdaki catch çalışır.

```
023 items = try JSONDecoder().decode([TodoItem].self, from: Data(contentsOf: self.fileURL))
```

Data(contentsOf:) dosyayı senkron okur; JSONDecoder baytları [TodoItem] dizisine çevirir. .self, dizi tipinin kendisini argüman olarak verir. try hem dosya okuma hem çözümleme hatasının catch'e gitmesini sağlar.

```
024 } catch {
```

Okuma ya da JSON çözümleme başarısız olduğunda bu kola geçilir. Hata kullanıcıya sade bir mesajla aktarılır; özgün Error ayrıntısı saklanmaz.

```
025 storageError = "Kaydedilen görevler okunamadı. Verilerinin üzerine yazmamak için uygulamayı yeniden başlatmayı dene."
```

Yükleme hatasını yayımlar. update metodu storageError nil değilken yazmayı reddettiği için boş listenin bozuk dosyanın üzerine yazılması engellenir. Yeniden başlatma, kalıcı olarak bozuk JSON'u kendiliğinden onarmaz.

026 } — 24. satırda açılan kapsamı kapatır: } catch {

027 } — 17. satırda açılan kapsamı kapatır: init(fileURL: URL? = nil) {

028 Boş satır — kod bölümlerini ayırır.

```
029 func add(_ title: String, important: Bool = false) {
```

Yeni görev ekleme API'sidir. İlk parametredeki `_`, çağırıcı `add("Başlık")` biçiminde okunur yapar; `important` için varsayılan `false` olduğundan normal görevde ikinci argüman gerekmez.

```
030 let title = title.trimmingCharacters(in: .whitespacesAndNewlines)
```

Başlığın başındaki ve sonundaki boşlukları ve satır sonlarını temizler. Ortadaki boşlukları silmez. Parametreyle aynı adlı yerel `let`, temizlenmiş değeri bundan sonra kullanır.

```
031 guard !title.isEmpty else { return }
```

Temizlenmiş metin boşsa işlemi erken bitirir. Kontrolün depoda bulunması, arayüz dışından yapılan eklemelerde de kuralın uygulanmasını sağlar.

```
032 update { $0.insert(TodoItem(title: title, isImportant: important), at: 0) }
```

`update`'e bir closure verir; `$0` değiştirilebilir görev dizisidir. Yeni `TodoItem` sıfıncı konuma eklenir, bu yüzden yeni görevler listenin başına gelir. Kaydetme ayrıntısı tek ortak metoda bırakılır.

```
033 } — 29. satırda açılan kapsamı kapatır: func add(_ title: String, important: Bool = false) {
```

```
034 Boş satır — kod bölümlerini ayırır.
```

```
035 func toggleCompletion(_ item: TodoItem) {
```

Verilen görevin tamamlanma durumunu tersine çevirecek işlemi tanımlar. Parametre değer kopyası olsa bile güncel kayıt aşağıda kimlik üzerinden bulunur.

```
036 update { items in
```

`update`'in sağladığı değiştirilebilir diziye `items` adı verir. Buradaki `items` closure parametresidir; doğrudan yayımlanan alanın üzerine yazılmaz.

```
037 guard let index = items.firstIndex(where: { $0.id == item.id }) else { return }
```

UUID eşleşmesiyle güncel dizindeki indeksi bulur. Bulunamazsa yalnızca bu closure'dan çıkar; `update` dış fonksiyonu devam edip değişmemiş diziyi yine kaydedebilir.

```
038 items[index].isCompleted.toggle()
```

`Bool.toggle()`, `false` değerini `true` veya `true` değerini `false` yapar. Yeni görev üretmek yerine mevcut kimlik korunur.

```
039 } — 36. satırda açılan kapsamı kapatır: update { items in
```

```
040 } — 35. satırda açılan kapsamı kapatır: func toggleCompletion(_ item: TodoItem) {
```

```
041 Boş satır — kod bölümlerini ayırır.
```

```
042 func toggleImportance(_ item: TodoItem) {
```

Önemli işaretini değiştirmek için ayrı bir API sunar. UI, dosya formatını bilmeden bu davranışı çağırabilir.

```
043 update { items in
```

Değişikliği ortak `update` yoluna iletir. Böylece önemli işareti de diğer işlemlerle aynı kayıt ve hata politikasını kullanır.

```
044 guard let index = items.firstIndex(where: { $0.id == item.id }) else { return }
```

Görevin başlığına değil id değerine bakarak indeksi bulur. Başlık değişse bile görev tanınır; bulunamama closure düzeyinde erken çıkıştır.

```
045 items[index].isImportant.toggle()
```

`isImportant` alanını tersine çevirir. Tamamlanma alanına dokunulmaz; tamamlanan bir görev önemli işaretini veri modelinde koruyabilir.

046 } — 43. satırda açılan kapsamı kapatır: update { items in

047 } — 42. satırda açılan kapsamı kapatır: func toggleImportance(_ item: TodoItem) {

048 Boş satır — kod bölümlerini ayırır.

```
049 func rename(_ item: TodoItem, to title: String) {
```

Görev başlığını değiştiren API'dir. to dış parametre etiketi rename(item, to: "Yeni") çağrısını okunur kılar; iç değişkenin adı title'dır.

```
050 let title = title.trimmingCharacters(in: .whitespacesAndNewlines)
```

Yeni başlığın çevresindeki boşlukları temizler. Ekleme ve yeniden adlandırmada aynı metin kuralı uygulanır.

```
051 guard !title.isEmpty else { return }
```

Boş başlıkla yeniden adlandırmayı reddeder. Geçerli eski başlık korunur; bu durumda bir storageError üretilmez.

```
052 update { items in
```

Başlık değişimini kayıt işlemini yöneten closure'a taşır. Aynı dosya yazma kodu her metoda kopyalanmaz.

```
053 guard let index = items.firstIndex(where: { $0.id == item.id }) else { return }
```

Güncel görevi kimliğinden bulur. Düzenleme ekranındaki TodoItem eski bir değer kopyası olsa bile doğru kayıt güncellenebilir.

```
054 items[index].title = title
```

Yalnızca title alanını değiştirir; id, tarih ve durum işaretleri korunur.

055 } — 52. satırda açılan kapsamı kapatır: update { items in

056 } — 49. satırda açılan kapsamı kapatır: func rename(_ item: TodoItem, to title: String) {

057 Boş satır — kod bölümlerini ayırır.

```
058 func delete(_ item: TodoItem) {
```

Görevi silen API'yi tanımlar. Dışarıdan TodoItem alınır; silme ölçütü bu öğenin kimliğidir.

```
059 update { $0.removeAll { $0.id == item.id } }
```

Dış \$0 görev dizisi, iç \$0 ise removeAll koşulunun incelediği tek görevdir. Kimliği eşleşen kayıtlar çıkarılır. Aynı sembolün iç içe closure'larda farklı değerleri temsil ettiğine dikkat et.

060 } — 58. satırda açılan kapsamı kapatır: func delete(_ item: TodoItem) {

061 Boş satır — kod bölümlerini ayırır.

```
062 private func update(_ mutation: (inout [TodoItem])) -> Void {
```

Ortak güncelleme mekanizmasıdır. mutation bir fonksiyon parametresidir; inout, closure'ın verilen dizi üzerinde değişiklik yapabilmesini sağlar. Void, closure'ın ayrıca sonuç değeri döndürmediğini belirtir.

```
063 guard storageError == nil else { return }
```

Depolama hatası varsa tüm değişiklikleri engeller. Hem okuma hem yazma hatasından sonra geçerlidir; mevcut uygulamada otomatik yeniden deneme veya hata sıfırlama akışı yoktur.

```
064 var updated = items
```

Mevcut dizinin değiştirilebilir değer kopyasını hazırlar. Swift dizileri copy-on-write kullanabilir; burada mantıksal amaç, kayıt başarılı olana kadar yayımlanan items değerini korumaktır.

```
065 mutation(&updated)
```

Closure'ı kopya üzerinde çalıştırır. & işareti, updated değişkeninin inout parametreye verildiğini gösterir.

```
066 do {
```

Dizin oluşturma ve dosya yazma işlemleri hata fırlatabildiği için do/catch kapsamına alınır.

```
067 try FileManager.default.createDirectory(at: fileURL.deletingLastPathComponent(),
withIntermediateDirectories: true)
```

Dosya adını çıkarıp üst dizini oluşturur. withIntermediateDirectories: true, eksik ara dizinlerin de oluşturulmasını ister. Var olan izin için yeniden kurulum gerekmez.

```
068 try JSONEncoder().encode(updated).write(to: fileURL, options: .atomic)
```

JSONEncoder diziyi JSON baytlarına çevirir; .atomic ile önce yardımcı dosyaya yazılıp hedefin değiştirilmesi istenir. Bu, yarım dosya riskini azaltır; şifreleme, yedekleme veya çok süreçli işlem kilidi değildir.

```
069 items = updated
```

Dosyaya yazma başarılı olduktan sonra yayımlanan listeyi günceller. Kullanıcının gördüğü veri ile son başarılı kayıt aynı değişikliği temsil eder.

```
070 } catch {
```

Dizin veya kayıt hatasını yakalar. items ataması bu noktaya gelmeden yapılmadığı için ekrandaki eski liste korunur.

```
071 storageError = "Değişiklik kaydedilemedi. Disk alanını ve dosya erişimini kontrol edip yeniden dene."
```

Yazma hatasını kullanıcıya duyurur. Mesaj yeniden denemeyi önerse de mevcut guard yüzünden aynı store örneğinde hata temizlenmeden yeni değişiklik yapılamaz; bu bir geliştirme noktasıdır.

```
072 } — 70. satırda açılan kapsamı kapatır: } catch {
```

```
073 } — 62. satırda açılan kapsamı kapatır: private func update(_ mutation: (inout [TodoItem]) -> Void) {
```

```
074 } — 12. satırda açılan kapsamı kapatır: final class TaskStore: ObservableObject {
```

7.3. ContentView.swift

ios_test/ContentView.swift

Filtre enum'u, yerel durum, yerleşim bileşenleri, etkileşimler ve Canvas özizlemesi. En uzun dosya olduğu için alt bloklar kaynak satırlarıyla ayrılmıştır.

Filtre türü ve görünüm durumu

```
001 import SwiftUI
```

SwiftUI görünüm tiplerini, yerleşim kaplarını ve durum sarmalayıcılarını kullanılabilir yapar. Dosyanın temel görevi arayüzü ve arayüze özgü geçici davranışları tarif etmektir.

002 Boş satır — kod bölümlerini ayırır.

```
003 private enum TaskFilter: String, CaseIterable, Identifiable {
```

Dosya kapsamındaki private enum, üç filtreyi sınırlı bir tür olarak tanımlar. String, görünen başlıkları rawValue yapar; CaseIterable tüm seçenekleri dolaştırır; Identifiable ForEach için kimlik sağlar.

```
004 case all = "Tüm görevler", important = "Önemli", completed = "Tamamlananlar"
```

Üç olası filtreyi ve Türkçe ham metinlerini tanımlar. Serbest metin yerine enum kullanmak yazım hatalı filtre değerlerini engeller; tüm durumlar switch içinde derleyici tarafından denetlenebilir.

```
005 var id: Self { self }
```

Kimlik olarak enum değerinin kendisini döndürür. Self mevcut tiptir; self mevcut değerdir. Üç durum birbirinden farklı olduğu için ayrıca UUID gerekmez.

```
006 var icon: String {
```

Filtrenin SF Symbols simge adını üreten hesaplanan özelliktir. Ayrı bir saklanan alan tutulmadığı için simge ile filtre arasında tutarsızlık oluşmaz.

```
007 switch self {
```

Mevcut enum değerine göre simge seçer. Bu switch bir ifade olarak String sonucu üretir; her case içinde ayrı return yazılmaz.

```
008 case .all: "square.stack.3d.up"
```

Tüm görevler için katmanlı kare simgesinin adını döndürür. Dize bir dosya yolu değil, sistem simgesi adıdır.

```
009 case .important: "star"
```

Önemli filtresinin simgesi yıldızdır. Aynı anlam görev satırındaki yıldızla da kullanılır.

```
010 case .completed: "checkmark.circle"
```

Tamamlananlar filtresinde onay işaretli daireyi kullanır. Switch tüm enum durumlarını kapsar; yeni case eklendiğinde buranın da güncellenmesi gerekir.

011 } — 7. satırda açılan kapsamı kapatır: switch self {

012 } — 6. satırda açılan kapsamı kapatır: var icon: String {

```
013 var subtitle: String {
```

Seçili filtresinin altında gösterilecek açıklamayı hesaplar. Başlık, simge ve açıklama aynı enum etrafında toplanmıştır.

```
014 switch self {
```

Filtre durumundan açıklama metnine geçişi yapar. if zinciri yerine enum üzerinde eksiksiz switch kullanılır.

```
015 case .all: "Zihnini boşalt. Bir sonraki adıma odaklan."
```

Tüm görevler ekranının alt başlığını döndürür.

```
016 case .important: "Senin için fark yaratacak işlere yer aç."
```

Önemli işler ekranına özgü açıklama döndürür; kullanıcıya listenin amacını anlatır.

```
017 case .completed: "Her küçük adım, bir ilerleme."
```

Tamamlanan işler ekranında ilerlemeyi vurgulayan metni döndürür.

```
018 } — 14. satırda açılan kapsamı kapatır: switch self {
```

```
019 } — 13. satırda açılan kapsamı kapatır: var subtitle: String {
```

```
020 } — 3. satırda açılan kapsamı kapatır: private enum TaskFilter: String, Caseltable, Identifiable {
```

```
021 Boş satır — kod bölümlerini ayırır.
```

```
022 struct ContentView: View {
```

ContentView bir SwiftUI View değeridir. struct olması görünüm tanımını hafif bir değer olarak tutar; kalıcı durumun yaşamı property wrapper'lar tarafından yönetilir.

```
023 @EnvironmentObject private var store: TaskStore
```

Üst görünümün sağladığı TaskStore'u tipine göre environment içinden okur. private dış erişimi sınırlar. Bu alan yeni depo yaratmaz; environmentObject sağlanmazsa çalışma zamanı hatası oluşur.

```
024 @State private var filter: TaskFilter = .all
```

Seçili filtre yerel arayüz durumudur ve başlangıçta all'dır. @State değiştiğinde bu değere bağımlı görünüm yeniden değerlendirilir; filtre JSON'a kaydedilmez.

```
025 @State private var search = ""
```

Arama metnini yerel durumda tutar. Boş dize arama kısıtı olmaması anlamına gelir; kullanıcı yazdıkça görünür liste hesaplaması değişir.

```
026 @State private var newTitle = ""
```

Yeni görev kutusunun taslak metnidir. Henüz TaskStore'a eklenmediği için yazılan her harf diske kaydedilmez.

```
027 @State private var editingItem: TodoItem?
```

Düzenlenen görevi optional olarak tutar. nil, açık düzenleme sayfası olmadığı anlamına gelir; bir TodoItem atanınca sheet(item:) bu seçimle açılır.

```
028 @State private var editedTitle = ""
```

Düzenleme sayfasının metin taslağıdır. Aynı state kullanmak, kullanıcı Vazgeç dediğinde asıl başlığın değişmemesini sağlar.

```
029 @FocusState private var composerFocused: Bool
```

Metin alanının klavye odağını izler ve değiştirebilir. @FocusState sıradan bir Bool'dan farklı olarak .focused bağlantısıyla gerçek odak sistemiyle eşleşir.

```
030 private let accent = Color(red: 0.36, green: 0.36, blue: 0.84)
```

Arayüzde tekrar kullanılan mor vurgu rengini tanımlar. Tek sabit kullanmak renk tutarlılığı sağlar; bu görünümde vurgu asset kataloğundan okunmaz.

031 Boş satır — kod bölümlerini ayırır.

```
032 private var completedCount: Int { store.items.filter(\.isCompleted).count }
```

Tamamlanmış görevleri key path olan `\.isCompleted` üzerinden filtreleyip sayar. Sonuç ayrı state değildir; görevler değiştiğinde güncel diziden türetilir.

```
033 private var visibleItems: [TodoItem] {
```

Ekranda gösterilecek görev dizisini üreten hesaplanan özelliştir. Filtrelenmiş ayrı bir kopyayı state olarak tutmayarak senkron tutma yükünden kaçınılır.

```
034 store.items.filter { item in
```

Depodaki her öğe için filtre closure'ını çalıştırır. `item`, sırayla incelenen `TodoItem` değeridir.

```
035 matches(item, filter: filter) && (search.isEmpty ||
item.title.localizedStandardContains(search))
```

Öğeler hem seçili filtreye uymalı hem arama boş olmalı veya başlık aramayı içermelidir. `&&` ve `||` kısa devre çalışır; `localizedStandardContains` kullanıcıya yönelik yerel ayarlı metin eşleştirmesi sağlar.

```
036 }.sorted { !$0.isCompleted && $1.isCompleted }
```

Tamamlanmamış öğeleri tamamlanmışların önüne sıralar. `$0` ve `$1` karşılaştırılan iki görevdir. Kural başlık veya `createdAt` tarihine göre sıralama yapmaz; yalnızca tamamlanma durumuna bakar.

037 } — 33. satırda açılan kapsamı kapatır: `private var visibleItems: [TodoItem] {`

038 Boş satır — kod bölümlerini ayırır.

Ana yerleşim ve düzenleme sayfası

```
039 var body: some View {
```

`View` protokolünün arayüz tanıımıdır. `some View` tek bir somut ama dışarıya gizli görünüm tipini ifade eder; herhangi bir tipin rastgele dönebilmesi anlamına gelmez.

```
040 HStack(spacing: 0) {
```

Ana `HStack`, kenar çubuğu ile içerik panelini yatay dizer. `spacing: 0`, paneller arasına otomatik boşluk eklenmesini engeller; ayırma işini `Divider` yapar. `VStack` kullanılsaydı paneller üst üste gelirdi.

```
041 sidebar
```

`sidebar` hesaplanan görünümünü ağaca ekler. Bu bölümlendirme `body`'yi okunur tutar; ayrı bir pencere ya da bağımsız veri deposu oluşturmaz.

```
042 Divider()
```

`HStack` içindeki `Divider` iki panel arasında dikey bir ayırıcı oluşturur. Yönü yerleşim bağlamıyla uyumludur.

```
043 VStack(alignment: .leading, spacing: 26) {
```

Ana paneldeki bölümleri yukarıdan aşağıya dizer. `leading`, içerikleri yatayda başlangıca hizalar; `spacing: 26` komşu bölümler arasındaki mesafedir, dış kenar boşluğu değildir.

```
044 header
```

Tarih, seçili filtre başlığı ve açıklamayı içeren header bölümünü ekler.

```
045 if let error = store.storageError {
```

Optional hata varsa güvenli biçimde error değişkenine açar. nil iken hata görünümü ağaca eklenmez; görünürlük için ayrı bir Bool tutulmaz.

```
046 Label(error, systemImage: "exclamationmark.triangle")
```

Hata mesajını uyarı simgesiyle birlikte Label içinde sunar. Label, metin ve simgeyi anlamlı bir UI bileşeni olarak birleştirir.

```
047 .font(.caption).foregroundColor(.red)
```

Hata yazısını küçük caption boyutunda ve kırmızı gösterir. Stil bu Label'a uygulanır; bütün ana paneli kırmızı yapmaz.

```
048 } — 45. satırda açılan kapsamı kapatır: if let error = store.storageError {
```

```
049 progressCard
```

Tüm görevlerden hesaplanan ilerleme kartını yerleştirir. Seçili liste veya arama sonucu bu kartın paydasını değiştirmez.

```
050 composer
```

Görev ekleme alanını yerleştirir. composer adı, taslak metnin girildiği arayüz parçasını temsil eder.

```
051 taskList
```

Listeyi veya boş durum görünümünü ekler. Bu bölüm esnek yükseklik alarak pencerenin kalan alanını doldurur.

```
052 footer
```

Bekleyen görev sayısı ve klavye ipucunun bulunduğu footer bölümünü ekler.

```
053 } — 43. satırda açılan kapsamı kapatır: VStack(alignment: .leading, spacing: 26) {
```

```
054 .padding(36)
```

Ana panel içeriği çevresine 36 punto boşluk ekler. Bu değer, VStack'in çocukları arasındaki 26 punto spacing'den farklı bir sorumluluğa sahiptir.

```
055 .frame(maxWidth: .infinity, maxHeight: .infinity)
```

Üst yerleşimin sunduğu kullanılabilir alana yayılmayı ister. infinity, sonsuz piksel üretmek değildir; ebeveynin boyut önerisi içindeki esnekliği ifade eder.

```
056 .background(Color(nsColor: .textBackgroundColor))
```

macOS'un metin arka planı rengini kullanır. nsColor köprüsü AppKit rengini SwiftUI Color'a çevirir; sistem rengi açık ve koyu görünümle uyumludur.

```
057 } — 40. satırda açılan kapsamı kapatır: HStack(spacing: 0) {
```

```
058 .tint(accent)
```

Alt kontrollerin vurgu rengini belirler. tint, özellikle düğme gibi buna uyan kontrollerin görünümüne yayılır; her foregroundStyle seçimini zorla değiştirmez.

```
059 .frame(minWidth: 800, minHeight: 580)
```

Arayüzün okunabilir kalması için minimum 800 × 580 punto boyut ister. Bu sabit boyut değildir; pencere daha büyük olabilir.

```
060 .sheet(item: $editingItem) { item in
```

`editingItem` binding'i nil değilken düzenleme sheet'i açılır. \$ işareti okuma-yazma bağlantısını verir; closure içindeki item düzenlenecek görevdir. `Identifiable` uyumu sheet'in ögeyi tanımasını sağlar.

```
061 VStack(alignment: .leading, spacing: 20) {
```

Düzenleme formunu soldan hizalı ve dikey düzenler. Başlık, giriş ve eylemler arasına 20 punto koyar.

```
062 Text("Görevi düzenle").font(.title2.bold())
```

Form başlığını `title2` boyutunda kalın gösterir; görsel hiyerarşide giriş alanından ayrılır.

```
063 TextField("Görev adı", text: $editedTitle)
```

`TextField`, `editedTitle` state'ine iki yönlü bağlanır. Kullanıcı yazınca state değişir; state değişince alan yeni değeri gösterir.

```
064 .textFieldStyle(.roundedBorder)
```

Yerel yuvarlatılmış kenarlıklı metin alanı stilini seçer. Form alanının düzenlenebilir olduğunu görsel olarak belirginleştirir.

```
065 .onSubmit { saveEdit(item) }
```

Metin alanından submit geldiğinde `saveEdit` çağrılır. Return ile kaydetme, düğme eylemiyle aynı iş mantığını paylaşır.

```
066 HStack {
```

Vazgeç ve Kaydet düğmelerini yatay yerleştirir. Eylemler tek satırda okunur ve formun altında birlikte durur.

```
067 Spacer()
```

Kullanılabilir yatay boşluğu alıp sonraki düğmeleri sağ tarafa iter. `Spacer`'ın etkisi bulunduğu `HStack`'in ana eksenindedir.

```
068 Button("Vazgeç") { editingItem = nil }.keyboardShortcut(.cancelAction)
```

`editingItem`'i nil yaparak sheet'i kapatır. Taslak ayrı tutulduğu için depodaki başlık değişmez; `cancelAction` macOS'ta standart iptal kısayoluna bağlanır.

```
069 Button("Kaydet") { saveEdit(item) }
```

Kaydet düğmesi, aynı `saveEdit` fonksiyonunu çağırır. Görev item ile, yeni başlık `editedTitle` ile belirlenir.

```
070 .buttonStyle(.borderedProminent)
```

Birincil eylemi dolgulu belirgin düğme stilinde sunar. Üst görünümünden gelen tint rengi burada kullanılabilir.

```
071 .disabled(editedTitle.trimmingCharacters(in: .whitespacesAndNewlines).isEmpty)
```

Yalnızca boşluklardan oluşan başlık için Kaydet'i devre dışı bırakır. UI doğrulamasıdır; `TaskStore.rename` içinde de ayrı koruma vardır.

```
072 .keyboardShortcut(.defaultAction)
```

Düğmeyi standart varsayılan eylem kısayoluna bağlar. Metin alanının `onSubmit` davranışı da aynı kaydetme fonksiyonuna yönelir.

```
073 } — 66. satırda açılan kapsamı kapatır: HStack {
```

```
074 } — 61. satırda açılan kapsamı kapatır: VStack(alignment: .leading, spacing: 20) {
```

```
075 .padding(28)
```

Sheet içeriğinin çevresine 28 punto boşluk ekler.

```
076 .frame(width: 400)
```

Düzenleme içeriğine 400 punto genişlik verir. Ana pencerenin boyutundan bağımsız, kısa bir başlık formu için kontrollü ölçü kullanılır.

```
077 } — 60. satırda açılan kapsamı kapatır: .sheet(item: $editingItem) { item in
```

```
078 } — 39. satırda açılan kapsamı kapatır: var body: some View {
```

```
079 Boş satır — kod bölümlerini ayırır.
```

Kenar çubuğu

```
080 private var sidebar: some View {
```

Kenar çubuğunu ayrı hesaplanan görünüm olarak tanımlar. `private` bu uygulama ayrıntısını `ContentView` dışında kullanıma kapatır.

```
081 VStack(alignment: .leading, spacing: 32) {
```

Marka, filtreler, motivasyon kartı ve gizlilik satırını dikey dizer. `leading`, çocukların başlangıç kenarını hizalar.

```
082 HStack(spacing: 12) {
```

Marka simgesi ile iki satırlık marka yazısını yatay yerleştirir. 12 punto aralık iki farklı görsel öğeyi birbirinden ayırır.

```
083 Image(systemName: "checkmark")
```

SF Symbols içindeki `checkmark` simgesini oluşturur. Harici bir resim dosyası gerektirmez.

```
084 .font(.system(size: 20, weight: .bold))
```

Sistem simgesinin 20 punto ve kalın ağırlıkla çizilmesini ister. SF Symbols, font ayarlarına uyum sağlar.

```
085 .foregroundColor(.white)
```

İşareti beyaz boyar; mor arka planda yeterli görsel ayrışma hedeflenir.

```
086 .frame(width: 40, height: 40)
```

Simgeye 40 × 40 punto yer ayırır. Sonraki background bu frame'in çevresinde çizildiği için küçük simgenin arkasında geniş bir kutu oluşur.

```
087 .background(accent.gradient, in: RoundedRectangle(cornerRadius: 13))
```

Vurgu renginin geçişini 13 punto köşe yarıçaplı kutuda çizer. `background` içerik arkasındadır; simgenin kendisini büyütmez.

```
088 VStack(alignment: .leading, spacing: 2) {
```

Marka adını ve sloganı alt alta, 2 punto aralıkla yerleştirir.

```
089 Text("odak").font(.system(size: 24, weight: .bold, design: .rounded))
```

odak adını 24 punto, kalın ve yuvarlatılmış font tasarımıyla gösterir. Bu tipografik seçim marka alanını diğer metinlerden ayırır.

```
090 Text("Daha az karmaşa.").font(.caption).foregroundColor(.secondary)
```

Sloganı daha küçük ve ikincil renkte gösterir; marka adıyla yarışmayan bir bilgi seviyesi oluşturur.

```
091 } — 88. satırda açılan kapsamı kapatır: VStack(alignment: .leading, spacing: 2) {
```

```
092 } — 82. satırda açılan kapsamı kapatır: HStack(spacing: 12) {
```

```
093 VStack(alignment: .leading, spacing: 8) {
```

Filtre başlığı ile seçenekleri dikey gruplar. Bu iç VStack'in spacing değeri dış kenar çubuğundan farklıdır.

```
094 Text("ÇALIŞMA ALANIM")
```

Filtre grubunun sabit başlığını gösterir. Bu satır etkileşimli bir kontrol değildir.

```
095 .font(.system(size: 10, weight: .semibold)).tracking(1.6)
```

Küçük, yarı kalın yazı ve 1,6 punto harf aralığı kullanır. tracking metnin harfleri arasındaki uzaklığı değiştirir.

```
096 .foregroundColor(.secondary).padding(.horizontal, 12).padding(.bottom, 6)
```

Başlığı ikincil renkte gösterip yatayda 12, altta 6 punto boşluk ekler. Başlığın seçenek satırlarıyla görsel hizasını düzenler.

```
097 ForEach(TaskFilter.allCases) { option in
```

CasIterable'dan gelen bütün filtreler için görünüm üretir. Identifiable sayesinde her seçenek kalıcı bir kimlikle izlenir; üç düğme elle tekrarlanmaz.

```
098 Button {
```

Her filtre için eylem ve label closure'ları ayrı olan bir Button başlatır. Böylece satırın tamamı düğme etiketi olarak tasarlanabilir.

```
099 withAnimation(.easeInOut(duration: 0.18)) { filter = option }
```

Seçili filtreyi 0,18 saniyelik easeInOut animasyon işlemi içinde değiştirir. Veriyi geciktirmez; bu işleminden doğan animasyon uygulanabilir görsel değişiklikleri yumuşatır.

```
100 } label: {
```

Düğmenin görünür içeriğini tanımlayan label closure'ına geçer. Önceki closure eylem, bu closure arayüz üretir.

```
101 HStack(spacing: 11) {
```

Simge, filtre metni, esnek boşluk ve sayacı yatay dizer. Bu bir menü satırı için HStack kullanımının tipik örneğidir.

```
102 Image(systemName: option.icon).frame(width: 18)
```

Filtrenin simgesini 18 punto genişlikte bir alana yerleştirir. Farklı simgelerin metin başlangıcını kaydırmasını azaltır.

```
103 Text(option.rawValue).fontWeight(filter == option ? .semibold : .regular)
```

Türkçe rawValue başlığını gösterir. Seçili satırı semibold, diğerlerini regular yaparak aktif durumu vurgular.

```
104 Spacer()
```

Başlık ile sayacı ayırıp sayacı satırın sonuna iter. Metnin uzunluğu değişse de sağ kenar düzenli kalır.

```
105 Text("\(store.items.filter { matches($0, filter: option) }.count)")
```

İlgili filtreye uyan tüm görevlerin sayısını string interpolation ile gösterir. \$0 incelenen görevdir; arama metni burada kullanılmadığı için sayaç arama sonucu sayısı değildir.

```
106 .font(.caption.monospacedDigit())
```

Caption boyutu ve eşit genişlikli rakamlar kullanır. Sayaç 1'den 8'e geçerken rakam genişliklerinin değişmesi azaltılır.

```
107 .foregroundColor(.secondary)
```

Sayacı ikincil renkte sunar; filtre başlığı daha baskın kalır.

108 } — 101. satırda açılan kapsamı kapatır: HStack(spacing: 11) {

```
109 .padding(12)
```

Filtre satırının içeriği çevresine 12 punto ekler. Hem nefes alanı hem daha geniş bir etkileşim bölgesi oluşur.

```
110 .foregroundColor(filter == option ? accent : Color.primary)
```

Seçili filtreyi mor, diğerlerini sistemin birincil rengiyle çizer. Üçlü koşul operatörü condition ? a : b iki stil arasından seçim yapar.

```
111 .background(filter == option ? accent.opacity(0.11) : .clear, in:
RoundedRectangle(cornerRadius: 10))
```

Seçili satırın arkasına yüzde 11 opaklıkta mor kutu, diğerlerine saydam arka plan koyar. Yuvarlatılmış köşeler seçimi bir bütün satır olarak gösterir.

```
112 .contentShape(Rectangle())
```

Hit-test şeklini dikdörtgen yapar. Etiketin boş bölgeleri dahil satırın tıklanabilir alanını belirgin bir şekle bağlar.

113 } — 100. satırda açılan kapsamı kapatır: } label: {

```
114 .buttonStyle(.plain)
```

Platformun standart düğme süslemesini kaldırır. Satırın seçili arka planı ve yazı stili zaten elle tasarlandığından çift görsel çerçeve oluşmaz.

115 } — 97. satırda açılan kapsamı kapatır: ForEach(TaskFilter.allCases) { option in

116 } — 93. satırda açılan kapsamı kapatır: VStack(alignment: .leading, spacing: 8) {

```
117 Spacer()
```

Dikey esnek boşluk bırakır; alt kart ve gizlilik bilgisini kenar çubuğunun altına iter. Bu kez Spacer, VStack içinde olduğu için dikey çalışır.

```
118 VStack(alignment: .leading, spacing: 9) {
```

Motivasyon kartının simge ve metinlerini dikey gruplar.

```
119 Image(systemName: "leaf").font(.title3).foregroundColor(accent)
```

Yaprak simgesini title3 ölçeğinde ve vurgu renginde gösterir. Dekoratif kartın görsel başlangıcıdır.

```
120 Text("Küçük adımlar,\nbüyük değişimler.")
```

\n kaçış dizisiyle metinde bilinçli satır kırılması yapar. Bu iki ayrı Text değil tek metin görünümüdür.

```
121 .font(.system(size: 15, weight: .medium)).lineSpacing(4)
```

15 punto medium yazı ve satırlar arasında ek 4 punto mesafe uygular. lineSpacing, tracking'den farklı olarak satırlar arasını etkiler.

```
122 Text("Bugün bir şeyle başla.")
```

Kartın ikinci kısa açıklamasını ekler.

```
123 .font(.caption).foregroundColor(.secondary)
```

İkinci açıklamayı caption ve secondary ile daha düşük hiyerarşide gösterir.

```
124 } — 118. satırda açılan kapsamı kapatır: VStack(alignment: .leading, spacing: 9) {
```

```
125 .padding(16)
```

Motivasyon kartının iç kenarlarına 16 punto boşluk ekler.

```
126 .frame(maxWidth: .infinity, alignment: .leading)
```

Kartı mevcut genişliğe yayar, içeriğini başlangıç tarafında tutar. leading, metnin kart içinde ortalanmasını engeller.

```
127 .background(accent.opacity(0.05), in: RoundedRectangle(cornerRadius: 14))
```

Kartın arkasına çok hafif mor ve 14 punto yuvarlatılmış köşeler ekler. Dekoratif arka plan iş mantığını değiştirmez.

```
128 Label("Sadece bu Mac'te saklanır", systemImage: "lock.shield")
```

Kilit-kalkan simgeli Label ile yerel saklama bilgisini kullanıcıya gösterir. Bu açıklama tek başına şifreleme veya erişim kontrolü uygulamaz.

```
129 .font(.system(size: 10)).foregroundColor(.secondary)
```

Gizlilik satırını küçük ve ikincil renkte çizer.

```
130 } — 81. satırda açılan kapsamı kapatır: VStack(alignment: .leading, spacing: 32) {
```

```
131 .padding(22)
```

Kenar çubuğunun çocuklarının çevresine 22 punto boşluk ekler.

```
132 .frame(width: 226)
```

Kenar çubuğu için 226 punto genişlik ayırır. Sabit sidebar ve esnek ana panel birlikte masaüstü düzeni oluşturur.

```
133 .frame(maxHeight: .infinity)
```

Kenar çubuğunu ebeveynin sunduğu yüksekliğe yayar; arka plan ve alt içerik tam panel boyunca yerleşir.

```
134 .background(.thinMaterial)
```

Sistemin ince materyal arka planını kullanır. Görünüm temaya ve arkasındaki içeriğe uyum gösterebilir; bu sıradan sabit RGB rengi değildir.

```
135 } — 80. satırda açılan kapsamı kapatır: private var sidebar: some View {
```

```
136 Boş satır — kod bölümlerini ayırır.
```

Başlık alanı

```
137 private var header: some View {
```

Ana başlık alanını ayrı bir görünüm olarak tanımlar. Tarih satırı ve filtre açıklaması birlikte yönetilir.

```
138 VStack(alignment: .leading, spacing: 18) {
```

Tarih satırıyla başlık grubunu 18 punto dikey aralıkla dizer.

```
139 HStack {
```

Tarih ve dekoratif güneş simgesini yatay yerleştirir.

```
140 Text(Date.now.formatted(Date.FormatStyle().day().month(.wide).weekday(.wide).locale(Locale(identifier: "tr_TR"))).uppercased())
```

Date.now değerini gün, uzun ay ve haftanın günü ile Türkçe biçimler, sonra büyük harfe çevirir. Bu bir zamanlayıcı değildir; görünüm yeniden değerlendirilmedikçe gece yarısında kendiliğinden güncelleme garantisi yoktur.

```
141 .font(.system(size: 10, weight: .semibold)).tracking(1.5).foregroundColor(.secondary)
```

Tarihi 10 punto yarı kalın, harf aralıklı ve ikincil renkte gösterir. Başlığa göre daha sessiz bir üst bilgi satırıdır.

```
142 Spacer()
```

Tarih ile simge arasındaki kalan genişliği doldurur; güneşi sağa iter.

```
143 Image(systemName: "sun.max").foregroundColor(.accent)
```

Sabit güneş simgesini vurgu rengiyle gösterir. Hava durumu verisi okumaz ve gündüz/gece hesabı yapmaz.

144 } — 139. satırda açılan kapsamı kapatır: HStack {

```
145 HStack(alignment: .bottom) {
```

Başlık grubunu yatay bir satıra alır ve çocukları alt kenarından hizalar. Burada ikinci öge Spacer olduğu için esas etki grubu sola tutmaktır.

```
146 VStack(alignment: .leading, spacing: 7) {
```

Büyük başlık ile alt açıklamayı 7 punto aralıkla alt alta koyar.

```
147 Text(filter.rawValue).font(.system(size: 32, weight: .bold, design: .rounded))
```

Seçili filtrenin Türkçe başlığını 32 punto kalın ve yuvarlatılmış fontla gösterir. filter değişimi otomatik olarak bu metne yansır.

```
148 Text(filter.subtitle).font(.subheadline).foregroundColor(.secondary)
```

Enum'un subtitle özelliğini subheadline ve secondary ile gösterir. Metin seçimi görünüm içinde tekrar switch yazmadan yapılır.

149 } — 146. satırda açılan kapsamı kapatır: VStack(alignment: .leading, spacing: 7) {

```
150 Spacer(minLength: 0)
```

Minimumu sıfır olan esnek alan ekler. Alan daraldığında zorunlu ek bir boşluk dayatmadan başlık grubunu başlangıca hizalamaya yardımcı olur.

151 } — 145. satırda açılan kapsamı kapatır: HStack(alignment: .bottom) {

152 } — 138. satırda açılan kapsamı kapatır: VStack(alignment: .leading, spacing: 18) {

153 } — 137. satırda açılan kapsamı kapatır: private var header: some View {

154 Boş satır — kod bölümlerini ayırır.

İlerleme kartı

```
155 private var progressCard: some View {
```

İlerleme kartını hesaplanan bir görünüm olarak ayırır. Halka, açıklama ve yüzde tek sorumluluk altında okunur.

```
156 HStack(spacing: 18) {
```

Halka, metin grubu ve yüzdeyi yatay dizer; aralarına 18 punto koyar.

```
157 ZStack {
```

ZStack şekilleri aynı alan üzerinde üst üste yerleştirir. Arka halka, ilerleme yayı ve onay simgesi aynı merkezde görünmelidir; HStack kullanmak onları yan yana dizerdi.

```
158 Circle().stroke(accent.opacity(0.12), lineWidth: 5)
```

Tam dairenin konturunu düşük opaklıkta çizer. Bu sabit arka halka, tamamlanmamış bölümü de görünür tutar.

```
159 Circle().trim(from: 0, to: store.items.isEmpty ? 0 : CGFloat(completedCount) /
CGFloat(store.items.count))
```

İkinci dairenin çizilecek kısmını 0 ile tamamlanan/toplam oranına sınırlar. Boş dizi için 0 seçilerek sıfıra bölme önlenir. CGFloat dönüşümü kesirli grafik hesabı içindir.

```
160 .stroke(accent, style: StrokeStyle(lineWidth: 5, lineCap: .round))
```

İlerleme yayına 5 punto kontur ve yuvarlak uç uygular. Dolgu yerine stroke kullanıldığı için dairenin içi kapatılmaz.

```
161 .rotationEffect(.degrees(-90))
```

Yayı eksi 90 derece döndürerek başlangıcını yukarı taşır. Döndürme bu ikinci daireye uygulanır; tüm kartı döndürmez.

```
162 Image(systemName: "checkmark").font(.system(size: 18, weight:
.semibold)).foregroundColor(accent)
```

Halkanın merkezine mor onay simgesini koyar. ZStack'in merkez hizası sayesinde ayrıca x/y koordinatı yazılmaz.

```
163 }.frame(width: 48, height: 48)
```

Katmanlı halka grubunu kapatıp 48 × 48 punto alana sınırlar. Bu ölçü, yanındaki metinle dengeli bir kart düzeni sağlar.

```
164 VStack(alignment: .leading, spacing: 5) {
```

Kart başlığını ve açıklamasını alt alta, başlangıca hizalı dizer.

```
165 Text(store.items.isEmpty ? "Yeni bir başlangıç" : "İlerleme kaydediyorsun")
```

Görev yokken başlangıç mesajı, görev varsa ilerleme mesajı seçer. Bu sadece sunum metni koşuludur.

```
166 .font(.system(size: 14, weight: .semibold))
```

Kart başlığını 14 punto yarı kalın yapar.

```
167 Text(store.items.isEmpty ? "İlk görevini ekle, gerisini adım adım hallet." :
"\(store.items.count) görevden \(completedCount) tanesi tamamlandı.")
```

Boş listeye yönlendirme metni, dolu listeye toplam ve tamamlanan sayısını gösterir. Hesap tüm store.items üzerinden yapıldığı için arama veya seçili filtreden bağımsızdır.

```
168 .font(.caption).foregroundColor(.secondary)
```

Açıklamayı caption ve secondary ile başlıktan ayırır.

```
169 } — 164. satırda açılan kapsamı kapatır: VStack(alignment: .leading, spacing: 5) {
```

```
170 Spacer()
```

Yüzde metnini kartın sağ kenarına iter.

```
171 Text(store.items.isEmpty ? "0%" : "\(completedCount * 100 / store.items.count)%")
```

Görev yoksa 0%, varsa tamsayı yüzdesi üretir. Int bölmesi ondalık kısmı atar; örneğin 1/3 için 33% görülür. Halka ise kesirli oranla çizilir.

```
172 .font(.system(size: 24, weight: .semibold, design: .rounded)).foregroundColor(accent)
```

Yüzdeyi 24 punto yarı kalın ve vurgu renginde gösterir.

```
173 } — 156. satırda açılan kapsamı kapatır: HStack(spacing: 18) {
```

```
174 .padding(20)
```

Kartın içeriği çevresine 20 punto boşluk ekler; arka plan sonraki satırda bu alanı da kapsar.

```
175 .background(accent.opacity(0.06), in: RoundedRectangle(cornerRadius: 16))
```

16 punto köşeli ve hafif mor arka plan çizer. Düşük opaklık sistemin açık/koyu zeminine uyum sağlar.

```
176 } — 155. satırda açılan kapsamı kapatır: private var progressCard: some View {
```

```
177 Boş satır — kod bölümlerini ayırır.
```

Yeni görev alanı

```
178 private var composer: some View {
```

Görev yazma alanını ayrı hesaplanan görünüm olarak tanımlar. Yerel newTitle durumu bu bölümün metin taslağıdır.

```
179 HStack(spacing: 12) {
```

Artı simgesi, metin alanı ve ekleme düğmesini aynı yatay satıra koyar.

```
180 Image(systemName: "plus.circle").font(.title3).foregroundColor(accent)
```

Artı daire simgesini title3 boyutunda ve vurgu renginde gösterir. Bu Image tek başına düğme değildir.

```
181 TextField("Aklında ne var? Yeni bir görev ekle...", text: $newTitle)
```

Görev taslağını newTitle binding'ine bağlar. Placeholder yalnızca alan boşken yol gösterir; varsayılan görev metni olarak kaydedilmez.

```
182 .textFieldStyle(.plain).focused($composerFocused).onSubmit(addTask)
```

Sade alan stili seçer, odak durumunu composerFocused ile bağlar ve submit olayını addTask'a yönlendirir. Aynı satırda üç modifier vardır; farklı sorumluluklara sahiptirler.

```
183 .accessibilityLabel("Yeni görev")
```

Erişilebilirlik teknolojilerine alanın adını Yeni görev olarak bildirir. Placeholder'ın görünümünden bağımsız açık bir kontrol adı sağlar.

```
184 Button(action: addTask) {
```

Düğmenin eylemine addTask fonksiyonunu verir. Parantezsiz fonksiyon adı burada çağrı değil, olay olduğunda çağrılacak fonksiyon değeridir.

```
185 Image(systemName: "arrow.up").fontWeight(.semibold).padding(5)
```

Ekleme düğmesinin etiketini yukarı ok olarak çizer ve 5 punto boşluk ekler. Simgeye semibold ağırlık uygular.

```
186 } — 184. satırda açılan kapsamı kapatır: Button(action: addTask) {
```

```
187 .buttonStyle(.borderedProminent).clipShape(RoundedRectangle(cornerRadius: 8))
```

Belirgin düğme stili seçer, ardından görünümü 8 punto köşeli şekle kırpır. clipShape, background gibi arka plana şekil eklemekten farklı olarak sınır dışını keser.

```
188 .disabled(newTitle.trimmingCharacters(in: .whitespacesAndNewlines).isEmpty)
```

Başlık yalnızca boşluklardan oluşuyorsa ekleme düğmesini kapatır. Klavyeden çağrı olabileceği için depodaki guard yine gereklidir.

```
189 .accessibilityLabel("Görev ekle").help("Görev ekle (Return)")
```

Simgeli düğmeye erişilebilir ad ve fare üstünde gösterilecek yardım metni ekler. Return desteği bir üst metin alanındaki onSubmit ile sağlanır.

```
190 } — 179. satırda açılan kapsamı kapatır: HStack(spacing: 12) {
```

```
191 .padding(14)
```

Görev giriş satırının çevresine 14 punto iç boşluk ekler.

```
192 .background(RoundedRectangle(cornerRadius: 12).strokeBorder(composerFocused ? accent :
Color.primary.opacity(0.12), lineWidth: 1))
```

Odak varken mor, yokken düşük opaklıklı çerçeve çizer. strokeBorder çizgiyi şeklin içinde tutar; görünümün tamamını doldurmaz.

```
193 .background {
```

Arka plana görünüm üreten ayrı bir closure ekler. Burada görsel dekor yerine kısayol alacak bir düğme yerleştirilmiştir.

```
194 Button("Yeni görev") { composerFocused = true }
```

Çağrılınca composerFocused değerini true yapar; .focused binding'i bunu klavye odağına taşır.

```
195 .keyboardShortcut("n", modifiers: .command).hidden()
```

Düğmeye Command+N kısayolu atar ve görünür etiketini gizler. hidden görünümü görünmez yapar; bu yöntemle kısayol tanımlanmıştır. Uygulama büyürse Commands içinde merkezî komut tanımı daha açık olabilir.

```
196 } — 193. satırda açılan kapsamı kapatır: .background {
```

```
197 } — 178. satırda açılan kapsamı kapatır: private var composer: some View {
```

```
198 Boş satır — kod bölümlerini ayırır.
```

Liste ve arama

```
199 private var taskList: some View {
```

Görev başlığı, arama, boş durum ve kaydırılabilir satırları bir araya getiren bölümdür.

```
200 VStack(spacing: 14) {
```

Üst araç satırı ile liste alanını 14 punto dikey aralıkla ayırır.

```
201 HStack {
```

Liste başlığı ile aramayı aynı yatay satırda tutar.

```
202 Text("GÖREVLER · \(visibleItems.count)")
```

Görünür görev sayısını gösterir. Sidebar sayaçlarından farklı olarak visibleItems kullandığı için arama metni bu sayıyı etkiler.

```
203 .font(.system(size: 10, weight: .semibold)).tracking(1.3).foregroundColor(.secondary)
```

Liste bölüm başlığı için küçük, harf aralıklı ve ikincil bir stil uygular.

```
204 Spacer()
```

Arama alanını sağ kenara iter ve başlıktan ayırır.

```
205 HStack(spacing: 6) {
```

Arama simgesi, alan ve koşullu temizleme düğmesini 6 punto yatay aralıkla dizer.

```
206 Image(systemName: "magnifyingglass").foregroundColor(.secondary)
```

Büyüteci ikincil renkte gösterir. Kullanıcıya alanın arama amaçlı olduğunu işaret eder.

```
207 TextField("Görev ara", text: $search).textFieldStyle(.plain)
```

TextField'ı search state'ine bağlar. Kullanıcı yazdığı anda visibleItems yeniden hesaplanır; ayrı bir Ara düğmesi yoktur.

```
208 .accessibilityLabel("Görev ara")
```

Arama alanının erişilebilir adını açıkça tanımlar.

```
209 if !search.isEmpty {
```

Arama metni varsa temizleme düğmesini görünüm ağacına ekler. Boşken gereksiz bir kontrol gösterilmez.

```
210 Button { search = "" } label: { Image(systemName: "xmark.circle.fill") }
```

Düğme search değerini boşaltır; etiketi dolu daire içindeki çarpıdır. Sonraki yeniden değerlendirmede arama kısıtı kalkar.

```
211 .buttonStyle(.plain).accessibilityLabel("Aramayı temizle")
```

Düğmeyi sade gösterir ve ekran okuyucu için Aramayı temizle adını verir.

212 } — 209. satırda açılan kapsamı kapatır: if !search.isEmpty {

```
213 }.font(.caption).frame(width: 160)
```

Arama grubunu kapatır, ortak fontu caption yapar ve 160 punto genişlik verir. Dar pencerelerde uzun arama metni bu sabit alan içinde düzenlenir.

214 } — 201. satırda açılan kapsamı kapatır: HStack {

```
215 if visibleItems.isEmpty {
```

Filtreleme ve aramadan sonra öge kalıp kalmadığını denetler. Bu koşul, tüm depoda hiç görev olmamasıyla aynı şey değildir.

```
216 VStack(spacing: 12) {
```

Boş durumdaki simge ve açıklamaları dikey ve ortalı düzenler.

```
217 Image(systemName: search.isEmpty ? "tray" : "magnifyingglass")
```

Arama boşsa tepsi, doluyrsa büyüteç gösterir. Boş liste ile sonuçsuz aramayı farklı görsel durumlar olarak anlatır.

```
218 .font(.system(size: 32, weight: .light)).foregroundColor(accent.opacity(0.7))
```

Boş durum simgesini 32 punto hafif ağırlık ve yüzde 70 opak vurgu rengiyle çizer.

```
219 Text(search.isEmpty ? "Burada henüz görev yok" : "Görev bulunamadı")
```

Arama durumuna göre henüz görev yok veya görev bulunamadı metni seçer.

```
220 .font(.headline)
```

Boş durum başlığını headline boyutunda sunar.

```
221 Text(search.isEmpty ? "Yeni bir görev ekle veya başka bir listeye göz at." : "Başka bir
kelimeyle aramayı dene.")
```

Kullanıcıya bir sonraki olası eylemi anlatır: görev eklemek/liste değiştirmek veya farklı kelime denemek.

```
222 .font(.caption).foregroundColor(.secondary)
```

Yardım metnini daha küçük ve ikincil renkte tutar.

```
223 }.frame(maxWidth: .infinity, maxHeight: .infinity)
```

Boş durum grubunu kullanılabilir alana yayar. Frame varsayılan merkez hizasıyla içeriğin liste alanında ortalanmasını sağlar.

```
224 } else {
```

Boş olmayan sonuçlar için alternatif görünüm dalını açar. SwiftUI ViewBuilder bu koşullu görünümü oluşturur.

```
225 ScrollView {
```

Varsayılan dikey ScrollView, çok sayıdaki görevin pencere dışına taşmak yerine kaydırılmasını sağlar. Tek başına tembel satır oluşturma sağlamaz.

```
226 LazyVStack(spacing: 8) {
```

LazyVStack satırları ihtiyaç oldukça üretir. ScrollView ile birlikte uzun listeler için uygundur; bu, TaskStore verisinin diskten parça parça yüklendiği anlamına gelmez.

```
227 ForEach(visibleItems) { item in taskRow(item) }
```

Her görünür görevi id kimliğiyle dolaşıp taskRow(item) üretir. Veri modelinin Identifiable uyumu burada doğrudan kullanılır.

```
228 }.padding(.vertical, 2)
```

LazyVStack'i kapatıp üst ve alta 2 punto boşluk ekler; satırlar arasındaki 8 punto spacing ayrı kalır.

229 } — 225. satırda açılan kapsamı kapatır: ScrollView {

230 } — 224. satırda açılan kapsamı kapatır: } else {

```
231 }.frame(maxHeight: .infinity)
```

Liste bölümünü kapatıp kalan yüksekliği almasına izin verir. Başlık, kart ve girişten sonra ana esnek alan burasıdır.

232 } — 199. satırda açılan kapsamı kapatır: private var taskList: some View {

233 Boş satır — kod bölümlerini ayırır.

Görev satırı

```
234 private func taskRow(_ item: TodoItem) -> some View {
```

Bir TodoItem için satır üreten yardımcı fonksiyondur. _ etiketsiz çağrıya izin verir; some View somut UI tipini dışarıya açmadan döndürür.

```
235 HStack(spacing: 13) {
```

Tamamlama düğmesi, başlık, yıldız ve menüyü yatay olarak dizer. 13 punto aralık kontrolleri ayırır; başlık esnek genişliği alır.

```
236 Button { withAnimation { store.toggleCompletion(item) } } label: {
```

Tamamlama düğmesi depodaki durumu animasyon işlemi içinde tersine çevirir. Dosya yazması yine senkrondur; withAnimation bir arka plan görevi başlatmaz.

```
237 Image(systemName: item.isCompleted ? "checkmark.circle.fill" : "circle")
```

Tamamlanan görevde dolu onaylı daire, diğerinde boş daire kullanır. Görsel durum doğrudan veri alanından türetilir.

```
238 .font(.system(size: 22, weight: .light))
```

Tamamlama simgesine 22 punto ve light ağırlık uygular.

```
239 .foregroundColor(item.isCompleted ? accent : Color.secondary.opacity(0.6))
```

Tamamlanan simgeyi vurgu rengine, diğerini silik ikincil renge boyar.

```
240 }.buttonStyle(.plain)
```

Tamamlama düğmesinin etiketini kapatır ve platform düğme çerçevesini kaldırır.

```
241 .accessibilityLabel(item.isCompleted ? "Görevi yeniden aç" : "Görevi tamamla")
```

Erişilebilir ad, mevcut durumu değil basılınca yapılacak eylemi anlatır: yeniden aç veya tamamla.

```
242 Text(item.title).font(.system(size: 14))
```

Görev başlığını 14 punto gösterir. Bu Text, düzenleme alanı değildir; düzenleme menüden sheet ile yapılır.

```
243 .strikethrough(item.isCompleted)
```

Görev tamamlanmışsa metnin üstünü çizir. Bool doğrudan stil koşulu olarak kullanılır.

```
244 .foregroundColor(item.isCompleted ? .secondary : .primary)
```

Tamamlanan başlığı ikincil renge düşürür, diğerini birincil renkte bırakır. Üst çizgiyle birlikte iki görsel ipucu sağlar.

```
245 .frame(maxWidth: .infinity, alignment: .leading)
```

Başlığın satırdaki boş genişliği almasını ve başlangıca hizalanmasını ister. Böylece yıldız ve menü sağda kalır; burada ayrıca Spacer gerekmez.

```
246 .textSelection(.enabled)
```

Başlık metninin kullanıcı tarafından seçilip kopyalanabilmesini etkinleştirir.

```
247 Button { store.toggleImportance(item) } label: {
```

Yıldız düğmesi önemli işaretini depoda değiştirir. Burada withAnimation kullanılmamıştır.

```
248 Image(systemName: item.isImportant ? "star.fill" : "star")
```

Önemli görev için dolu yıldız, diğerleri için kontur yıldız gösterir.

```
249 .foregroundColor(item.isImportant ? Color.orange : Color.secondary.opacity(0.55))
```

Önemli yıldız turuncu, diğerini silik ikincil renkte çizir. Durum, yalnızca simgenin doluluğuyla sınırlı kalmaz.

```
250 }.buttonStyle(.plain).accessibilityLabel(item.isImportant ? "Önemli işaretini kaldır" : "Önemli olarak işaretle")
```

Yıldız düğmesini sade yapar ve duruma göre anlamlı eylem adı verir. Bu tek satır hem closure kapanışı hem iki modifier içerir.

```
251 Menu {
```

Göreve ait ikincil eylemleri açılır Menu içinde toplar. Her satırda ayrı Düzenle ve Sil düğmeleri göstermeyerek yer tasarrufu sağlar.

```
252 Button("Düzenle", systemImage: "pencil") { editedTitle = item.title; editingItem = item }
```

Önce düzenleme taslağına başlığı kopyalar, sonra editingItem atayarak sheet'i açar. Noktalı virgül aynı satırdaki iki ifadeyi ayırır; sıralama formun doğru metinle açılmasını sağlar.

```
253 Button("Sil", systemImage: "trash", role: .destructive) { withAnimation { store.delete(item) } }
```

Silme düğmesine destructive rolü verir ve depodan siler. Rol görsel/anlamsal ipucudur; otomatik onay penceresi veya geri alma davranışı eklemes.

```
254 } label: { Image(systemName: "ellipsis").foregroundColor(.secondary) }
```

Menünün görünür etiketini üç nokta simgesi olarak tanımlar. Önceki closure menü seçenekleri, bu closure tetikleyici görünümüdür.

```
255 .menuStyle(.borderlessButton).menuIndicator(.hidden).fixedSize()
```

Kenarlıksız menü stili seçer, varsayılan göstergiyi gizler ve ideal boyutu korur. Amaç üç noktanın yanında ikinci bir açılır menü oku görünmemesidir.

```
256 .accessibilityLabel("Görev seçenekleri")
```

Simgeli menünün erişilebilir adını Görev seçenekleri yapar.

```
257 } — 235. satırda açılan kapsamı kapatır: HStack(spacing: 13) {
```

```
258 .padding(16)
```

Görev satırının çevresine 16 punto ekler. Satır arka planı bu iç boşluğu da kapsayacaktır.

```
259 .background(Color.primary.opacity(item.isCompleted ? 0.015 : 0.035), in:
RoundedRectangle(cornerRadius: 12))
```

Tamamlanan görevlerde daha silik bir arka plan seçer ve 12 punto köşeler kullanır. Color.primary temaya göre değiştiği için arka plan açık/koyu görünümle uyum sağlar.

```
260 } — 234. satırda açılan kapsamı kapatır: private func taskRow(_ item: TodoItem) -> some View {
```

```
261 Boş satır — kod bölümlerini ayırır.
```

Alt bilgi ve eylem yardımcıları

```
262 private var footer: some View {
```

Alt bilgi satırını hesaplanan görünüm olarak tanımlar. Görev sayısı ve kısayol ipucu burada toplanır.

```
263 HStack {
```

Durum noktası, bekleyen görev metni ve kısayolu yatay dizer.

```
264 Circle().fill(Color.green).frame(width: 5, height: 5)
```

5 × 5 punto yeşil bir daire çizer. Bu dekoratif nokta gerçek zamanlı disk/bağlantı sağlık kontrolüne bağlı değildir.

```
265 Text("\(store.items.count - completedCount) görev seni bekliyor")
```

Toplamdan tamamlananları çıkararak bekleyen görev sayısını gösterir. Seçili filtre ve arama burada da hesaba katılmaz.

```
266 Spacer()
```

Kısayol ipucunu alt satırın sağ kenarına taşır.

```
267 Text("Yeni görev %N")
```

Yeni görev için Command+N ipucunu gösterir. Bu Text tek başına kısayol oluşturmaz; gerçek tanım composer içindeki düğmededir.

```
268 }.font(.system(size: 10)).foregroundColor(.secondary)
```

Alt satırı kapatır; çocuklara ortak 10 punto font ve ikincil renk uygular. Çocukta açıkça verilmiş yeşil renk gibi stiller kendi görünümünde geçerlidir.

269 } — 262. satırda açılan kapsamı kapatır: private var footer: some View {

270 Boş satır — kod bölümlerini ayırır.

```
271 private func matches(_ item: TodoItem, filter: TaskFilter) -> Bool {
```

Bir görevin belirtilen filtreye uyup uymadığını döndüren yardımcı fonksiyondur. Sidebar sayaçları ile görünür listenin aynı kuralı paylaşmasını sağlar.

```
272 switch filter {
```

Filtre enum'una göre Bool sonucu üretir; tüm seçenekler açıkça ele alınır.

```
273 case .all: true
```

Tüm görevler filtresi her öğeyi kabul eder; tamamlanmış görevler de buna dahildir.

```
274 case .important: item.isImportant && !item.isCompleted
```

Önemli filtresi hem yıldızlı hem tamamlanmamış öğeleri alır. Yıldızlı görev tamamlanınca işareti silinmez, yalnızca bu listeden çıkar.

```
275 case .completed: item.isCompleted
```

Tamamlananlar filtresi yalnızca isCompleted true olanları kabul eder.

276 } — 272. satırda açılan kapsamı kapatır: switch filter {

277 } — 271. satırda açılan kapsamı kapatır: private func matches(_ item: TodoItem, filter: TaskFilter) -> Bool {

278 Boş satır — kod bölümlerini ayırır.

```
279 private func addTask() {
```

Arayüzün ekleme akışını tanımlar. TaskStore veriyi doğrulayıp kaydederken bu fonksiyon taslak, arama ve odak gibi UI durumunu yönetir.

```
280 store.add(newTitle, important: filter == .important)
```

Başlığı depoya gönderir. Seçili filtre important ise yeni görev de önemli oluşturulur; diğer filtrelerde false gider.

```
281 guard store.storageError == nil else { return }
```

Depolama hatası varsa taslağı temizlemeden çıkar. Kullanıcının yazdığı metnin kaybolmamasını sağlar. add metodunun sessizce reddettiği boş başlık için ayrı bir başarı sonucu yoktur.

```
282 newTitle = ""
```

Hata yoksa yeni görev taslağını boşaltır. Ekleme sonrası kullanıcı yeni bir başlık yazabilir.

```
283 if filter == .completed { filter = .all }
```

Tamamlananlar ekranından görev eklendiyse all'a geçer. Yeni görev tamamlanmamış olduğundan aksi hâlde mevcut filtrede görünmezdi.

```
284 search = ""
```

Aramayı temizler. Yeni başlığın önceki arama ifadesine uymadığı için gizlenmesini önler.

```
285 composerFocused = true
```

Klavye odağını tekrar giriş alanına verir. Art arda görev ekleme akışını hızlandırır.

```
286 } — 279. satırda açılan kapsamı kapatır: private func addTask() {
```

```
287 Boş satır — kod bölümlerini ayırır.
```

```
288 private func saveEdit(_ item: TodoItem) {
```

Düzenleme formunun kaydetme eylemidir. Depo işlemi ile sheet kapatma davranışını birleştirir.

```
289 store.rename(item, to: editedTitle)
```

Seçili görevi editedTitle taslağıyla yeniden adlandırır. Boşluk temizleme ve dosya kaydı TaskStore'da yapılır.

```
290 if store.storageError == nil { editingItem = nil }
```

Depolama hatası yoksa editingItem'i nil yaparak sheet'i kapatır. rename bir başarı Bool'u döndürmediğinden boş başlığın reddedilmesi gibi sessiz durumlar burada ayırt edilmez.

```
291 } — 288. satırda açılan kapsamı kapatır: private func saveEdit(_ item: TodoItem) {
```

```
292 } — 22. satırda açılan kapsamı kapatır: struct ContentView: View {
```

```
293 Boş satır — kod bölümlerini ayırır.
```

Xcode önizlemesi

```
294 #Preview {
```

Xcode Canvas için bir önizleme tanımlar. Bu uygulamanın @main giriş noktası değildir; tasarım sırasında ContentView'i ayrı bağlamda görmeyi sağlar.

```
295 ContentView()
```

Önizleme için ContentView örneği oluşturur. Aşağıdaki environmentObject bağımlılığı yine gereklidir.

```
296 .environmentObject(TaskStore(fileURL:
URL.temporaryDirectory.appendingPathComponent("odak-preview.json")))
```

Önizlemeye geçici dizinde dosya kullanan TaskStore verir. Gerçek görev dosyasını ayırır; ancak ad sabit olduğu için önizleme oturumları aynı geçici dosyayı paylaşabilir, otomatik temiz başlangıç garantisi yoktur.

```
297 .environment(\.locale, Locale(identifier: "tr_TR"))
```

Önizlemede Türkçe locale seçer; çalışan uygulamadaki environment ayarıyla aynı niyeti taşır.

```
298 } — 294. satırda açılan kapsamı kapatır: #Preview {
```

7.4. ios_testTests.swift

ios_testTests/ios_testTests.swift

Geçici dosyalar üzerinde model davranışı ve bozuk veri korumasını sınan Swift Testing testleri.

```
001 import Foundation
```

Foundation; URL, UUID, Data ve FileManager gibi dosya/test yardımcılarını sağlar.

```
002 import Testing
```

Swift Testing çatısını içe aktarır. @Test, #expect ve #require bu modülden gelir; bu dosya XCTestCase kullanmaz.

```
003 @testable import ios_test
```

Uygulama modülünü test edilebilir erişimle içe aktarır. Test derlemesinde internal bildirimlere erişilebilir; private alanları sınırsızca açmaz.

004 Boş satır — kod bölümlerini ayırır.

```
005 @MainActor
```

Testleri MainActor bağlamına yerleştirir. TaskStore da bu aktöre izole edildiği için çağrılar aynı bağlamda yapılır.

```
006 struct ios_testTests {
```

Bir Swift Testing test grubudur. XCTestCase kalıtımı gerekmez; @Test ile işaretlenmiş fonksiyonlar keşfedilir.

```
007 @Test func tasksPersistAcrossLaunches() throws {
```

Görevlerin yeniden yüklenmesini sınanan testtir. throws, dosya hatalarının veya #require başarısızlığının testi başarısız bitirebilmesini sağlar.

```
008 let directory = URL.temporaryDirectory.appendingPathComponent(UUID().uuidString)
```

Her test çalıştırmasına benzersiz geçici dizin ayırır. UUID, paralel ya da tekrar eden testlerin aynı dosyaya çarpışmasını azaltır.

```
009 defer { try? FileManager.default.removeItem(at: directory) }
```

defer, fonksiyondan çıkılırken temizliği çalıştırır; test erken hata verse de kapsamdan çıkışta işler. try? temizlik hatasını bilerek yok sayar.

```
010 let url = directory.appendingPathComponent("tasks.json")
```

Geçici dizine tasks.json dosya adını ekler. Bu işlem henüz dosya oluşturmaz.

```
011 let store = TaskStore(fileURL: url)
```

Bağımlılık enjeksiyonuyla test deposunu gerçek kullanıcı dosyasından ayırır.

```
012 store.add("  ")
```

Sadece boşluk içeren başlık eklemeyi dener; doğrulamanın UI'dan bağımsız çalışmasını sınar.

```
013 #expect(store.items.isEmpty)
```

Boş başlığın listeye eklenmediğini doğrular. #expect başarısızsa bulguyu kaydeder; testi mutlaka hemen durdurmak zorunda değildir.

```
014 store.add(" İlk görev ", important: true)
```

Çevresinde boşluk olan önemli görev ekler. Tek işlemle başlık temizliği ve önemli alanın başlangıç değeri için veri hazırlanır.

```
015 let item = try #require(store.items.first)
```

İlk öğenin varlığını zorunlu kılar ve optional'dan çıkarır. #require başarısızsa devam eden adımlar anlamsız olacağı için test throws yoluyla durur.

```
016 #expect(item.title == "İlk görev")
```

Başlığın dış boşluklarının temizlendiğini doğrular.

```
017 store.toggleCompletion(item)
```

Görevi tamamlanmış duruma geçirir. Ardından yeni store ile bu değişikliğin kalıcılığı kontrol edilecektir.

```
018 store.rename(item, to: "Yeni başlık")
```

Aynı görevin başlığını değiştirir. item eski bir değer kopyasıdır; depo id ile bulduğu için işlem çalışır.

```
019 let restored = TaskStore(fileURL: url)
```

Aynı dosyayı okuyan yeni TaskStore yaratır. Bu gerçek uygulama sürecini yeniden başlatmaz; yeniden yükleme davranışını model düzeyinde simüle eder.

```
020 #expect(restored.items.first?.title == "Yeni başlık")
```

Yeniden yüklenen başlığın değişmiş olduğunu doğrular. ?. optional chaining, öge yoksa nil üreterek karşılaştırmanın başarısız olmasını sağlar.

```
021 #expect(restored.items.first?.isCompleted == true)
```

Tamamlanma durumunun diskte korunduğunu sınar.

```
022 #expect(restored.items.first?.isImportant == true)
```

Önemli işaretinin diskte korunduğunu sınar. toggleImportance davranışı bu testte ayrıca çalıştırılmamıştır.

```
023 restored.delete(item)
```

Yeniden yüklenen depodan görevi siler. Eski item'ın kimliği korunmuş olduğu için doğru kayıt bulunur.

```
024 #expect(TaskStore(fileURL: url).items.isEmpty)
```

Bir kez daha dosyayı okuyup silmenin kalıcı olduğunu doğrular. Sadece bellekte boşalma kontrolüyle yetinilmez.

```
025 } — 7. satırda açılan kapsamı kapatır: @Test func tasksPersistAcrossLaunches() throws {
```

```
026 Boş satır — kod bölümlerini ayırır.
```

```
027 @Test func corruptStorageIsNotOverwritten() throws {
```

Bozuk dosyanın üzerine yazılmamasını sınavan ikinci testtir. Bu veri koruma davranışı, hata mesajının yalnızca görünmesinden daha önemlidir.

```
028 let url = URL.temporaryDirectory.appendingPathComponent(UUID().uuidString)
```

Benzersiz geçici dosya URL'si oluşturur. Uzantı olmaması sorun değildir; JSON çözümleyici dosya uzantısına değil içeriğine bakar.

```
029 defer { try? FileManager.default.removeItem(at: url) }
```

Test sonunda bozuk örnek dosyayı kaldırır; temizlik hatasını test sonucuna katmaz.

```
030 let original = Data("broken data".utf8)
```

Geçerli JSON olmayan UTF-8 baytları oluşturur. Data, string yerine dosyaya yazılacak ham baytları temsil eder.

```
031 try original.write(to: url)
```

Bozuk içeriği dosyaya yazar. try, hazırlık başarısızsa testin devam etmesini önler.

```
032 let store = TaskStore(fileURL: url)
```

Depo dosyayı okumayı ve çözmeyi dener; beklenen sonuç storageError oluşmasıdır.

```
033 #expect(store.storageError != nil)
```

Okuma/çözümleme hatasının kaydedildiğini doğrular. Mesaj metninin tam içeriğine bağımlı kalmaz.

```
034 store.add("Yeni görev")
```

Hatalı depo durumunda yeni görev eklemeyi dener. update'in koruma guard'ı bu değişikliği engellemelidir.

```
035 #expect(try Data(contentsOf: url) == original)
```

Dosya baytlarını tekrar okuyup orijinal bozuk veriyle karşılaştırır. Dosyanın gerçekten korunmasını sınar; sadece items boş kaldı demek yeterli olmazdı.

```
036 } — 27. satırda açılan kapsamı kapatır: @Test func corruptStoragesIsNotOverwritten() throws {
```

```
037 } — 6. satırda açılan kapsamı kapatır: struct ios_testTests {
```

7.5. ios_testUITests.swift

ios_testUITests/ios_testUITests.swift

XCTest hazırlık kancaları, temel açılış ve açılış performansı ölçümü.

```
001 //
```

Yorum satırı: derlenip çalıştırılmaz; geliştiriciye bağlam veya şablon yönergesi verir.

```
002 // ios_testUITests.swift
```

Yorum satırı: derlenip çalıştırılmaz; geliştiriciye bağlam veya şablon yönergesi verir.

```
003 // ios_testUITests
```

Yorum satırı: derlenip çalıştırılmaz; geliştiriciye bağlam veya şablon yönergesi verir.

```
004 //
```

Yorum satırı: derlenip çalıştırılmaz; geliştiriciye bağlam veya şablon yönergesi verir.

```
005 // Created by Bugra on 25.09.2026.
```

Dosyanın oluşturulma bilgisini taşıyan Xcode şablon yorumudur; çalışma zamanı davranışını etkilemez.

```
006 //
```

Yorum satırı: derlenip çalıştırılmaz; geliştiriciye bağlam veya şablon yönergesi verir.

007 Boş satır — kod bölümlerini ayırır.

```
008 import XCTest
```

XCTest, UI otomasyonu ve performans ölçümü API'lerini sağlar. Model testlerindeki Swift Testing'den ayrı bir test çatısıdır.

009 Boş satır — kod bölümlerini ayırır.

```
010 final class ios_testUITests: XCTestCase {
```

XCTestCase'ten türeyen UI test sınıfıdır. final, alt sınıf oluşturulmasını engeller; test ile başlayan uygun metotlar XCTest tarafından keşfedilir.

011 Boş satır — kod bölümlerini ayırır.

```
012 override func setUpWithError() throws {
```

Her testten önce çalışacak hazırlık metodunu override eder. throws, hazırlık sırasında hata oluşabileceğini ifade eder.

```
013 // Put setup code here. This method is called before the invocation of each test method in the class.
```

Yorum satırı: derlenip çalıştırılmaz; geliştiriciye bağlam veya şablon yönergesi verir.

014 Boş satır — kod bölümlerini ayırır.

```
015 // In UI tests it is usually best to stop immediately when a failure occurs.
```

Yorum satırı: derlenip çalıştırılmaz; geliştiriciye bağlam veya şablon yönergesi verir.

```
016 continueAfterFailure = false
```

Bir UI doğrulaması başarısız olursa testin devam etmemesini ister. Yanlış ekran durumunda sonraki adımların anlamsız hatalar üretmesini azaltır.

017 Boş satır — kod bölümlerini ayırır.

```
018 // In UI tests it's important to set the initial state - such as interface orientation -
    required for your tests before they run. The setUp method is a good place to do this.
```

Yorum satırı: derlenip çalıştırılmaz; geliştiriciye bağlam veya şablon yönergesi verir.

```
019 } — 12. satırda açılan kapsamı kapatır: override func setUpWithError() throws {
```

```
020 Boş satır — kod bölümlerini ayırır.
```

```
021 override func tearDownWithError() throws {
```

Her testten sonra çalışacak temizlik kancasını tanımlar. Şu an gövdede yalnızca şablon yorumu vardır; gerçek temizleme işlemi yapılmaz.

```
022 // Put teardown code here. This method is called after the invocation of each test method in
    the class.
```

Yorum satırı: derlenip çalıştırılmaz; geliştiriciye bağlam veya şablon yönergesi verir.

```
023 } — 21. satırda açılan kapsamı kapatır: override func tearDownWithError() throws {
```

```
024 Boş satır — kod bölümlerini ayırır.
```

```
025 @MainActor
```

UI test metodunu ana aktöre izole eder; UI otomasyon API'lerinin beklediği bağlama uyum sağlar.

```
026 func testExample() throws {
```

Temel açılış testi iskeletidir. throws hata aktarımına izin verir; içinde işlevsel doğrulama assertion'ı bulunmaz.

```
027 // UI tests must launch the application that they test.
```

Yorum satırı: derlenip çalıştırılmaz; geliştiriciye bağlam veya şablon yönergesi verir.

```
028 let app = XCUIApplication()
```

Test hedefindeki uygulamayı temsil eden XCUIApplication otomasyon nesnesini oluşturur. Bu, SwiftUI App nesnesinin kendisi değildir.

```
029 app.launch()
```

Uygulamayı test otomasyonu üzerinden başlatır. Görev ekleme veya filtre doğrulaması yapılmaz; açılışın kendisi denir.

```
030 Boş satır — kod bölümlerini ayırır.
```

```
031 // Use XCTAssert and related functions to verify your tests produce the correct results.
```

Şablon, işlevsel assertion eklemeyi önerir; yorumun kendisi doğrulama yapmaz ve bu metotta ek bir assertion yoktur.

```
032 } — 26. satırda açılan kapsamı kapatır: func testExample() throws {
```

```
033 Boş satır — kod bölümlerini ayırır.
```

```
034 @MainActor
```

Performans testini MainActor bağlamına taşır.

```
035 func testLaunchPerformance() throws {
```

Uygulama açılış süresini ölçen test metodudur. İşlevsel doğruluk ile performans farklı test amaçlarıdır.

```
036 if #available(macOS 10.15, iOS 13.0, tvOS 13.0, watchOS 7.0, *) {
```

Ölçüm API'sinin işletim sistemi uygunluğunu denetler. Şablondaki iOS/tvOS/watchOS listesi uygulamanın bu platformları desteklediğini göstermez; proje hedefi macOS'tur.

```
037 // This measures how long it takes to launch your application.
```

Yorum satırı: derlenip çalıştırılmaz; geliştiriciye bağlam veya şablon yönergesi verir.

```
038 measure(metrics: [XCTApplicationLaunchMetric()]) {
```

XCTApplicationLaunchMetric ile uygulama açılış metriğini toplar. measure bloğu ölçüm düzenine göre birden fazla kez yürütülebilir.

```
039 XCUIApplication().launch()
```

Her ölçüm yürütmesinde uygulamayı başlatır. Bu satır herhangi bir görev sonucunu assert etmez.

```
040 } — 38. satırda açılan kapsamı kapatır: measure(metrics: [XCTApplicationLaunchMetric()]) {
```

```
041 } — 36. satırda açılan kapsamı kapatır: if #available(macOS 10.15, iOS 13.0, tvOS 13.0, watchOS 7.0, *) {
```

```
042 } — 35. satırda açılan kapsamı kapatır: func testLaunchPerformance() throws {
```

```
043 } — 10. satırda açılan kapsamı kapatır: final class ios_testUITests: XCTestCase {
```

7.6. ios_testUITestsLaunchTests.swift

ios_testUITests/ios_testUITestsLaunchTests.swift

UI yapılandırmaları için uygulamayı başlatıp test raporuna ekran görüntüsü ekleyen test.

```
001 //
```

Yorum satırı: derlenip çalıştırılmaz; geliştiriciye bağlam veya şablon yönergesi verir.

```
002 // ios_testUITestsLaunchTests.swift
```

Yorum satırı: derlenip çalıştırılmaz; geliştiriciye bağlam veya şablon yönergesi verir.

```
003 // ios_testUITests
```

Yorum satırı: derlenip çalıştırılmaz; geliştiriciye bağlam veya şablon yönergesi verir.

```
004 //
```

Yorum satırı: derlenip çalıştırılmaz; geliştiriciye bağlam veya şablon yönergesi verir.

```
005 // Created by Bugra on 25.09.2026.
```

Dosyanın oluşturulma bilgisini taşıyan Xcode şablon yorumudur; çalışma zamanı davranışını etkilemez.

```
006 //
```

Yorum satırı: derlenip çalıştırılmaz; geliştiriciye bağlam veya şablon yönergesi verir.

007 Boş satır — kod bölümlerini ayırır.

```
008 import XCTest
```

Açılış otomasyonu ve ekran görüntüsü eki için XCTest'i içe aktarır.

009 Boş satır — kod bölümlerini ayırır.

```
010 final class ios_testUITestsLaunchTests: XCTestCase {
```

Açılış senaryolarını içeren ayrı XCTestCase sınıfını tanımlar. Test gruplarını amaçlarına göre ayırır.

011 Boş satır — kod bölümlerini ayırır.

```
012 override class var runsForEachTargetApplicationUIConfiguration: Bool {
```

Testi hedefin UI yapılandırmaları için çalıştırma politikasını override eder. class var, örnek yerine sınıf düzeyinde ayardır.

```
013 true
```

Politikayı etkinleştirir. Gerçekte hangi yapılandırmaların kullanılacağı test koşumunun yapılandırmasına bağlıdır; burada tek tek tema listelenmez.

014 } — 12. satırda açılan kapsamı kapatır: override class var runsForEachTargetApplicationUIConfiguration: Bool {

015 Boş satır — kod bölümlerini ayırır.

```
016 override func setUpWithError() throws {
```

Her test öncesi hazırlık kancasını açar.

```
017 continueAfterFailure = false
```

İlk başarısızlıktan sonra UI testinin sürmesini engeller.

018 } — 16. satırda açılan kapsamı kapatır: override func setUpWithError() throws {

019 Boş satır — kod bölümlerini ayırır.

```
020 @MainActor
```

Açılış testini ana aktöre yerleştirir.

```
021 func testLaunch() throws {
```

Uygulamayı başlatıp görüntü ekleyen test metodudur. Ekrandaki görevlerin doğruluğunu sınavan bir assertion içermez.

```
022 let app = XCUIApplication()
```

Test edilen uygulama için otomasyon temsilcisi oluşturur.

```
023 app.launch()
```

Uygulamayı başlatır. Sonraki screenshot çağrısı bu çalışan uygulamadan görüntü alacaktır.

024 Boş satır — kod bölümlerini ayırır.

```
025 // Insert steps here to perform after app launch but before taking a screenshot,
```

Şablon, görüntü almadan önce uygulama içi hazırlık eklenebileceğini söyler. Mevcut test burada ek adım uygulamaz.

```
026 // such as logging into a test account or navigating somewhere in the app
```

Şablon, görüntü almadan önce uygulama içi hazırlık eklenebileceğini söyler. Mevcut test burada ek adım uygulamaz.

027 Boş satır — kod bölümlerini ayırır.

```
028 let attachment = XCTAttachment(screenshot: app.screenshot())
```

Uygulama ekran görüntüsünü XCTest eki haline getirir. README'deki örnek görselleri üretmek için kullanılan geçici render yöntemiyle aynı akış değildir.

```
029 attachment.name = "Launch Screen"
```

Test raporunda ekin okunabilir adını Launch Screen yapar.

```
030 attachment.lifetime = .keepAlways
```

Test başarılı olsa da ekin korunmasını ister. Varsayılan yalnızca hata odaklı saklama davranışına güvenilmez.

```
031 add(attachment)
```

Eki mevcut test sonucuna ekler. Dosyayı doğrudan docs/screenshots klasörüne yazmaz; Xcode sonuç paketinde görünür.

032 } — 21. satırda açılan kapsamı kapatır: func testLaunch() throws {

033 } — 10. satırda açılan kapsamı kapatır: final class ios_testUITestsLaunchTests: XCTestCase {

8. Xcode ayarları ve yardımcı dosyalar

project.pbxproj, Xcode projesinin hedeflerini, dosya gruplarını, derleme aşamalarını ve ayarlarını tutar. Buradaki uzun kimlikler proje nesnelerini birbirine bağlar; iş mantığı değildir. Genellikle Xcode üzerinden değiştirilir.

Uygulama, model testleri ve UI testleri ayrı hedeflerdir; Debug ve Release için tekrar eden ayarlar bulunması bu yüzden normaldir.

Ayar / dosya	Mevcut değer veya rol	Geliştirici için anlamı
SDKROOT	macosx	macOS SDK'sı ile derlenir.
SUPPORTED_PLATFORMS	macosx	Proje adı iOS desteği anlamına gelmez.
MACOSX_DEPLOYMENT_TARGET	14.0	Desteklenen en eski macOS sürümüdür; kullanılan SDK sürümüyle aynı kavram değildir.
SWIFT_VERSION	5.0	Swift dil uyumluluk modu; derleyici sürüm numarası değildir.
PRODUCT_BUNDLE_IDENTIFIER	test-app-ident.ios-test	Uygulamayı tanımlayan teknik kimliktir; test hedefleri ayrı sonekler taşır.
CODE_SIGN_STYLE	Automatic	İmzalama yönetimi Xcode'a bırakılır; dağıtım hazır sertifika bulunduğunu garanti etmez.
GENERATE_INFOPLIST_FILE	YES	Uygulama metadata dosyası derleme ayarlarından üretilir.
ASSETCATALOG_COMPILER_APPICON_NAME	AppIcon	Uygulama simgesi olarak kullanılacak asset setini seçer.
ASSETCATALOG_COMPILER_GLOBAL_ACCENT_COLOR_NAME	AccentColor	Global vurgu asset adıdır; ContentView ayrıca kendi mor rengini tanımlar.
Assets.xcassets	Asset kataloğu	Simge ve renk kaynaklarını Xcode'a tanıtır.
Preview Content	Önizleme kaynak grubu	Tasarım sırasında kullanılabilecek asset alanıdır; görev deposu değildir.
README.md	Proje giriş belgesi	Kurulum ve kullanım anlatır; bu PDF kod öğretime odaklanır.
docs/screenshots	İki tema görseli	Gerçek SwiftUI görünümünden örnek görevlerle alınmış render çıktılarıdır.

8.1. AppIcon Contents.json nasıl okunur?

AppIcon.appiconset/Contents.json dosyasındaki images dizisi, macOS simgesi için çeşitli boyut ve ölçekleri eşler. Her kayıttaki filename PNG dosyasını, idiom: mac hedef ailesini, size mantıksal boyutu, scale ise piksel ölçeğini belirtir. Örneğin size: 16x16 ve scale: 2x, 32 × 32 piksel görsele karşılık gelir. Bu nedenle aynı piksel boyutunun farklı mantıksal kullanım rolleri olabilir.

16, 32, 128, 256 ve 512 punto boyutları için 1x ve 2x kayıtlar bulunur. info içindeki author: xcode, asset metadata üreticisini; version: 1 ise bu metadata biçiminin sürümünü ifade eder. Bunlar uygulamanın ürün sürüm numarası değildir. Binary PNG dosyaları kod satırı olarak açıklanamaz; katalog içindeki rolleri açıklanır.

8.2. .gitignore dosyasının beş satırı

Satır	Desen	Neden dışarıda tutulur?
1	.DS_Store	Finder'ın klasör görünüm metadata'sı kaynak kod değildir.
2	DerivedData/	Xcode'un yeniden üretilebilir ara derleme verileri.
3	build/	Yerel derleme çıktıları.
4	*.xcuserstate	Kullanıcıya özgü IDE pencere/oturum durumu.
5	xcuserdata/	Geliştiriciye özgü Xcode kullanıcı ayarları.

.gitignore, daha önce Git tarafından izlenen dosyaları kendiliğinden takipten çıkarmaz; esas olarak izlenmeyen eşleşen dosyaların eklenmesini engeller. Projedeki paylaşılan yapılandırmalar ile kişisel IDE tercihlerini ayırmak amaçlanır.

9. Testleri nasıl yorumlamalı?

Bir testin amacı “uygulama açıldı” ile “doğru çalışıyor” ifadelerini ayırmaktır. Model testleri UI’yı açmadan dosya ve veri davranışını sınar. UI testleri ise çalışan uygulamayı dışarıdan yönetir. Bu projede birinci grup daha somut davranış kontrolleri içerirken ikinci grup ağırlıklı Xcode şablonunun açılış ve ölçüm akışıdır.

Davranış	Mevcut kanıt	Sınır
Boş görev reddi	items.isEmpty beklentisi	Boş yeniden adlandırma ayrıca sinanmıyor.
Başlık temizliği	İlk görev başlığı karşılaştırması	Tüm Unicode boşluk durumları taranmıyor.
JSON kalıcılığı	Yeni TaskStore ile yeniden okuma	Gerçek süreç kapat/aç otomasyonu değil.
Tamamlama / düzenleme / silme	Yeniden yüklenen veri beklentileri	Bütün bulunamayan-kimlik durumları yok.
Önemli veri alanı	important: true sonrası yeniden okuma	toggleImportance ayrı test edilmemiş.
Bozuk veri koruması	Dosya baytlarının eşitliği	Yazma hatası ve disk doluluğu simülasyonu yok.
UI açılışı	XCUUIApplication.launch	Görev ekleme, filtre, sheet ve arama assertion'ları yok.
Açılış performansı	XCTApplicationLaunchMetric	Kullanıcı etkileşim gecikmesini ölçmez.

`#expect`, beklenen koşulun doğru olup olmadığını kaydeder. `#require` ise bir sonraki adımın devam edebilmesi için gerekli koşulu zorunlu kılar; örneğin ilk görev yoksa onun başlığını sınamak anlamsızdır. Swift Testing’in test tanımlama biçimi kaynak [8]’de açıklanır.

Geçici URL enjeksiyonu, test edilebilirliğin en önemli küçük tasarım kararlarından biridir. TaskStore(fileURL:) sayesinde test kendi dosyasını yaratabilir, bozuk veri koyabilir ve sonunda temizleyebilir. Bunun için ağ servisi, sahte UI veya karmaşık dependency injection çatısı gerekmez.

10. Öğrenmeyi pekiştiren uygulamalar

Bu alıştırmalar öneridir; projeye uygulanmadı. Önce beklenen davranışı yaz, sonra kodu değiştir ve son olarak ilgili davranışı doğrula. Görsel değişikliklerde Canvas veya çalışan uygulama, veri davranışlarında model testi daha uygun geri bildirim verir.

10.1. Yerleşim deneyi

Görev satırındaki HStack'i geçici olarak VStack yap. Dört öğenin neden artık tek satır oluşturmadığını gözlemler. Sonra HStack'e dönüp spacing değerini 13'ten 24'e çıkar. padding aynı kalırken yalnızca çocuklar arası uzaklığın değiştiğini açıkla. Beklenen sonuç: Stack yönü geometrik ilişkiyi, spacing iç öğeler arasını, padding bütün satırın çevresini etkiler.

10.2. Yeni bir filtre

TaskFilter'a pending durumu ekleyip başlık, icon ve subtitle switch'lerini tamamla. matches içinde !item.isCompleted dön. ForEach(allCases) sayesinde düğmenin neden kendiliğinden üretildiğini açıkla. Beklenen sonuç: enum merkezli tasarım, seçenekleri tek yerde tanımlamaya yardım eder; iş kuralı yine matches içinde eklenmelidir.

10.3. Önemli işaretinin kalıcılığı

Geçici URL'li TaskStore oluştur, görev ekle, toggleImportance çağır ve aynı URL'den yeni store aç. isImportant değerini doğrula. İkinci kez değiştirip false olduğunun da kalıcı kaldığını sınavabilirsiniz. Beklenen sonuç: yalnızca UI yıldızını değil, gerçek dosya üzerinden davranışı doğrulamak.

10.4. Hata sonrası yeniden deneme tasarımı

Yazma hatasının storageError nedeniyle sonraki işlemleri engellediğini izle. Güvenli bir yeniden deneme API'sinin ne zaman hatayı temizleyeceğini tasarla. Okuma hatasında boş items listesini hemen kaydetmenin neden tehlikeli olduğunu açıkla. Beklenen sonuç: hata mesajını silmek ile verinin yeniden güvenli yazılabilir olması aynı şey değildir.

10.5. UI testini büyütmek

Önce testler için yalıtılmış bir görev dosyası sağla. Ardından erişilebilir adlarla Yeni görev alanına metin yaz, Görev ekle düğmesine bas ve başlığın görüldüğünü doğrula. Sonra tamamla ve Tamamlananlar filtresini kontrol et. Beklenen sonuç: açılış testini gerçek kullanıcı senaryosuna dönüştürmek; mevcut kullanıcı verisini test verisiyle karıştırmamak.

10.6. Kısa öz değerlendirme

- HStack içinde Spacer neden dikey boşluk üretmez? Çünkü ana eksen yataydır.
- newTitle neden TaskStore.items içinde tutulmaz? Henüz kaydedilmemiş bir UI taslağıdır.
- Önemli görev tamamlanınca yıldız neden veri modelinden silinmez? Filtre koşulu tamamlanmış dışlar; toggleCompletion yalnızca isCompleted alanını değiştirir.

- MainActor neden dosya yazmayı hızlandırmaz? İzolasyon sağlar; senkron I/O'yu otomatik arka plan işi yapmaz.
- .atomic neden yedek değildir? Yazma stratejisini düzenler; eski sürümleri arşivlemez.
- Önizleme neden environmentObject ister? ContentView'in bağımlılığı önizlemede de aynıdır.
- sheet neden nil ile kapanır? Sunum durumu optional editingItem binding'ine bağlıdır.
- createdAt varken neden liste tarihe göre sıralanmaz? Mevcut sorted karşılaştırıcısı sadece isCompleted alanını kullanır.

11. Resmî kaynaklar ve belge üretimi

Anlatımın birincil kaynağı bu deponun Swift dosyalarıdır. Aşağıdaki bağlantılar API kavramlarını derinleştirmek içindir. İnceleme tarihi: 25 Eylül 2026. API sayfalarının bazıları JavaScript gerektirdiğinden otomatik erişimde yalnızca referans kabuğu görüntülenebilmiştir; proje davranışı doğrudan yerel koddan çıkarılmıştır.

[1] Apple — HStack: <https://developer.apple.com/documentation/swiftui/hstack>

[2] Apple — LazyVStack: <https://developer.apple.com/documentation/swiftui/lazyvstack>

[3] Apple — StateObject: <https://developer.apple.com/documentation/swiftui/stateobject>

[4] Apple — Managing model data in your app:
<https://developer.apple.com/documentation/swiftui/managing-model-data-in-your-app>

[5] Swift — Closures: <https://docs.swift.org/swift-book/documentation/the-swift-programming-language/closures/>

[6] Apple — Eliminate data races using Swift Concurrency:
<https://developer.apple.com/videos/play/wwdc2022/110351/>

[7] Apple — Data writing option, atomic:
<https://developer.apple.com/documentation/foundation/nsdata/writingoptions/atomic>

[8] Swift — Defining test functions: <https://docs.swift.org/latest/documentation/testing/definingtests/>

11.1. PDF'nin nasıl üretildiği

Metin ve satır açıklamaları docs/education altında düzenlenebilir kaynak olarak tutulur. build_guide.py, kaynak satırlarını numaralarıyla okuyup açıklamalarla eşler; eksik açıklama varsa üretimi durdurur. PDF, Python ve ReportLab ile oluşturulur. Türkçe karakterler için TrueType fontlar PDF'ye gömülür. Pypdf ile sayfa/metin kontrolü, görsel olarak da örnek sayfa incelemesi yapılır. Bu PDF üretim araçları uygulamanın çalışma zamanı bağımlılığı değildir.

Bu belge eğitim amaçlı bir kaynak kod incelemesidir. Uygulama davranışında değişiklik yapmaz. Ekran görselleri mevcut SwiftUI arayüzünün örnek görevlerle oluşturulan açık ve koyu tema çıktılarıdır; kullanıcı görevlerini temsil etmez.

11.2. Kaynak anlık görüntüsü ve kapsam kontrolü

Aşağıda kaynak dosyaların satır sayıları ve SHA-256 özetlerinin ilk 16 karakteri yer alır. Tam özetler docs/education/source_snapshot.json dosyasındadır. Kod değişirse açıklamalar gözden geçirilmelidir.

ios_testApp.swift · 29 satır · 824e9c4ea9cc7deb

TaskStore.swift · 74 satır · f266990028a4aed8

ContentView.swift · 298 satır · 5ec6f43265cbba8b

ios_testTests.swift · 37 satır · 7e6a1a53373fb1e5

ios_testUITests.swift · 43 satır · 7189fa927b1e7531

ios_testUITestsLaunchTests.swift · 33 satır · ac6389e0b74c40b4